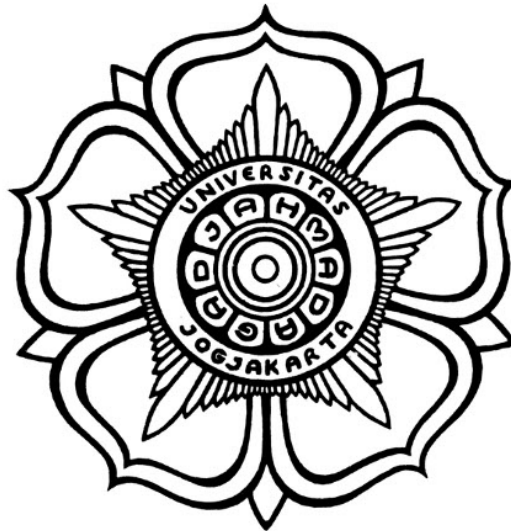


**MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022
SECURE CODING UNTUK PENINGKATAN DAN KONVERSI
VERSI WEB APP DENGAN MEMANFAATKAN AI DAN
PERANGKAT OPENSOURCE.**

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

Muhammmad Farrel Akbar
22/492806/TK/53947

**PROGRAM SARJANA PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
«TAHUN PENDADARAN»**

HALAMAN PENGESAHAN

MANAJEMEN PERANGKAT LUNAK STANDAR ISO27001/2022 SECURE CODING UNTUK PENINGKATAN DAN KONVERSI VERSI WEB APP DENGAN MEMANFAATKAN AI DAN PERANGKAT OPENSOURCE.

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Farrel Akbar
22/492806/TK/53947

Telah disetujui dan disahkan

Pada tanggal

Dosen Pembimbing I

Dosen Pembimbing II

Dani Adhipta, S.Si., M.T.
«NIP xxxxxx»

Addin Suwastono, Ir., S.T., M.Eng., IPM.
«NIP xxxxxx»

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Muhaammad Farrel Akbar
NIM : 22/492806/TK/53947
Tahun terdaftar : 2022
Program : Sarjana
Program Studi : Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Muhammad Farrel Akbar
22/492806/TK/53947

HALAMAN PERSEMBAHAN

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

KATA PENGANTAR

Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Dani Adhipta, S.Si., M.T. yang telah membimbing dan memberikan arahan serta ilmu yang sangat berharga selama penyusunan skripsi ini.
2. Addin Suwastono, Ir., S.T., M.Eng., IPM. yang telah membimbing dan memberikan arahan serta ilmu yang sangat berharga selama penyusunan skripsi ini.
3. Orang tua saya, Bapak Andrian Nugroho dan Ibu Retny Anggeriana yang telah memberikan dukungan moril dan materiil serta doa yang tiada henti sehingga saya dapat menyelesaikan studi ini.

Saya sebagai penulis menyadari bahwa dalam penyusunan skripsi ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi kesempurnaan skripsi ini. Semoga skripsi ini dapat bermanfaat bagi pembaca pada umumnya dan bagi penulis pada khususnya. Amin.

Yogyakarta, 13 Maret 2017

Muhammad Farrel Akbar
22/492806/TK/53947

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xiii
ABSTRACT	xiv
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Batasan Penelitian	5
1.5 Manfaat Penelitian	6
1.5.1 Manfaat Akademis	6
1.5.2 Manfaat Praktis.....	6
1.6 Sistematika Penulisan.....	6
BAB II Tinjauan Pustaka dan Dasar Teori	8
2.1 Tinjauan Literatur	8
2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL.....	8
2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan	8
2.1.3 Transformasi Kode dengan Bantuan LLM	9
2.1.4 Pendekatan Neuro-Simbolik untuk Deteksi Kerentanan Keamanan	9
2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis	10
2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi.....	10
2.1.7 Ringkasan Perbandingan.....	11
2.2 Dasar Teori	12
2.2.1 PHP dan Perubahan Antar Versi	12
2.2.2 ISO/IEC 27001:2022 dan <i>Secure Coding</i>	13
2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP	13
2.2.4 DevSecOps dan Integrasi Keamanan dalam Pipeline Pengembang-	
an.....	14
2.2.5 <i>Abstract Syntax Trees</i> dan <i>Static Analysis</i>	14
2.2.6 Rector: Alat Transformasi Kode Berbasis AST	15

2.2.7	PHPStan: Alat Analisis Statis untuk PHP	15
2.2.8	Semgrep: Alat Analisis Statis Ringan untuk Keamanan	15
2.2.9	Model-Model LLM untuk Analisis Kode	16
2.2.10	Ollama: Menerapkan LLM Secara Lokal	17
BAB III Metodologi Penelitian		18
3.1	Alat dan Bahan Penelitian	18
3.1.1	Perangkat Keras	18
3.1.2	Perangkat Lunak	18
3.1.3	Model LLM yang Dievaluasi	19
3.1.4	Bahan Penelitian	19
3.2	Metode Penelitian	20
3.3	Arsitektur Sistem	21
3.3.1	Scanner (scanner.py)	22
3.3.2	Converter (converter.py)	22
3.3.3	Analyzer (analyzer.py)	22
3.3.4	AI Engine (ai_engine.py)	23
3.3.5	ISO Mapper (iso_mapper.py)	23
3.3.6	Main (main.py)	23
3.4	Alur Data Pipeline	24
3.5	Desain Prompt LLM	24
3.5.1	Struktur Prompt	25
3.5.2	Keputusan Desain Prompt	25
3.6	Desain Pemetaan ISO/IEC 27001:2022	26
3.7	Dataset Studi Kasus	27
3.8	Rancangan Eksperimen Komparasi Model LLM	27
3.8.1	Kondisi Eksperimen	27
3.8.2	Metrik Evaluasi	28
3.8.3	Interpretasi Trade-off Antar Metrik	28
3.9	Keamanan dan Privasi Data	28
3.10	Metode Pengujian dan Evaluasi	28
BAB IV Hasil dan Diskusi		30
4.1	Hasil Konversi Rector	30
4.1.1	Contoh Transformasi Rector	31
4.1.2	File yang Gagal Dikonversi	32
4.2	Hasil Pemindaian Keamanan Semgrep	33
4.2.1	Pre-Scan: Baseline Keamanan Kode Asli	33
4.2.2	Post-Scan: Perbandingan Setelah Konversi	33
4.3	Hasil Validasi PHPStan	33
4.4	Status Kepatuhan ISO/IEC 27001:2022	34

4.5	Hasil Komparasi Lima Model LLM (Kondisi A)	35
4.5.1	Format Compliance Rate	35
4.5.2	Syntax Validity Rate	36
4.5.3	Statistik Ringkasan.....	37
4.5.4	Analisis Kualitatif: Perbandingan Respons terhadap Temuan SQL Injection	37
4.5.5	Trade-off dan Implikasinya	40
4.6	Diskusi dan Keterbatasan	40
4.6.1	Keterbatasan Dataset	40
4.6.2	Keterbatasan Teknis Pipeline	41
4.6.3	Interpretasi Confidence Score	41
BAB V	Tambahan (Opsional)	42
BAB VI	Kesimpulan dan Saran	43
6.1	Kesimpulan	43
6.2	Saran	43
DAFTAR PUSTAKA		44

DAFTAR TABEL

Tabel 2.1	Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian	11
Tabel 2.2	Perbandingan Penelitian Terdahulu dan Aspek Metodologi.....	12
Tabel 3.1	Pemetaan argumen <code>-target-php</code> ke konfigurasi Rector	22
Tabel 3.2	Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A	27
Tabel 4.1	Status kepatuhan ISO/IEC 27001:2022 hasil <i>pipeline</i> pada dataset DTI UGM.....	35

DAFTAR GAMBAR

Gambar 3.1	Alur penelitian dengan model prototipe	20
Gambar 3.2	Arsitektur sistem <i>pipeline</i> dengan enam modul Python.....	21
Gambar 3.3	Alur data <i>pipeline</i> dari <i>input</i> hingga laporan akhir	24
Gambar 4.4	Ringkasan hasil konversi Rector terhadap 245 <i>file</i> PHP dataset DTI UGM	30
Gambar 4.5	Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A	34
Gambar 4.6	<i>Format compliance rate</i> kelima model LLM pada Kondisi A (<i>zero-shot</i> , <i>prompt</i> identik)	36
Gambar 4.7	<i>Syntax validity rate</i> dan <i>code block extraction rate</i> kelima model LLM	36
Gambar 4.8	Statistik lengkap komparasi kelima model LLM pada Kondisi A .	37

DAFTAR SINGKATAN

[SAMPLE]

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
$\mathbf{w}$	=	vektor bobot
$\mathbf{x}$	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Intisari ditulis menggunakan bahasa Indonesia dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Intisari sekurang-kurangnya berisi tentang latar belakang dan tujuan penelitian, metodologi yang digunakan, hasil penelitian, kesimpulan dan implikasi, dan Kata kunci yang berhubungan dengan penelitian.

Kata Kunci ditulis maksimal 5 kata yang paling berhubungan dengan isi skripsi. Silakan mengacu pada ACM / IEEE *Computing classification* jika Anda adalah mahasiswa Sarjana TI <http://www.acm.org/about/class/> atau mengacu kepada IEEE keywords http://www.ieee.org/documents/taxonomy_v101.pdf jika Anda berasal dari Prodi Sarjana TE.

Kata kunci : Kata kunci 1, Kata kunci 2, Kata kunci 3, Kata kunci 4, Kata kunci 5

Contoh Abstrak Teknik Elektro:

"Penelitian ini bertujuan untuk mengembangkan sistem pengendalian suhu ruangan dengan menggunakan microcontroller. Metodologi yang digunakan adalah desain sirkuit, implementasi sistem pengendalian, dan pengujian performa. Hasil penelitian menunjukkan bahwa sistem pengendalian suhu ruangan yang dikembangkan mampu mengendalikan suhu ruangan dengan akurasi sebesar $\pm 0,5^{\circ}\text{C}$. Kesimpulan dari penelitian ini adalah sistem pengendalian suhu ruangan yang dikembangkan efektif dan efisien.

Kata kunci: microcontroller, sistem pengendalian suhu, akurasi."

Contoh Abstrak Teknik Biomedis:

"Penelitian ini bertujuan untuk mengevaluasi keefektifan prototipe alat pemantau denyut jantung berbasis elektrokardiogram (ECG) untuk pasien jantung. Metodologi yang digunakan meliputi desain dan pembuatan prototipe, pengujian dengan pasien, dan analisis data. Hasil penelitian menunjukkan bahwa prototipe alat pemantau denyut jantung berbasis ECG memiliki akurasi yang baik dan mampu memantau denyut jantung pasien secara efektif. Kesimpulan dari penelitian ini adalah prototipe alat pemantau denyut jantung berbasis ECG merupakan solusi yang efektif dan efisien untuk memantau pasien jantung.

Kata kunci: elektrokardiogram, alat pemantau denyut jantung, akurasi."

Contoh Abstrak Teknologi Informasi:

"Penelitian ini bertujuan untuk mengevaluasi keamanan dan privasi pengguna aplikasi media sosial terpopuler. Metodologi yang digunakan meliputi analisis kebijakan privasi dan pengaturan keamanan, pengujian penetrasi, dan survei pengguna. Hasil penelitian menunjukkan bahwa beberapa aplikasi media sosial memiliki kebijakan privasi yang kurang jelas dan rendahnya tingkat keamanan. Kesimpulan dari penelitian ini adalah pentingnya meningkatkan kebijakan privasi dan tingkat keamanan pada aplikasi media sosial untuk melindungi privasi dan data pengguna.

Kata kunci: media sosial, keamanan, privasi, pengguna."

ABSTRACT

Abstract ditulis italic (miring) menggunakan bahasa Inggris dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Abstract adalah versi Bahasa Inggris dari intisari. Abstract dapat ditulis dalam beberapa paragraf. Baris pertama paragraph harus menjorok ke dalam sekitar 1 cm. Tidak disarankan menggunakan mesin penerjemah melainkan tulis ulang.

Keywords : Keyword 1, Keyword 2, Keyword 3, Keyword 4, Keyword 5

BAB I

PENDAHULUAN

1.1 Latar Belakang

PHP bukan bahasa yang didesain untuk menjadi bahasa pemrograman besar. Rasmus Lerdorf memperkenalkannya pada tahun 1995 sebagai kumpulan skrip sederhana untuk mengelola halaman web pribadinya [1]. Tapi dalam waktu beberapa tahun, PHP berkembang jauh melampaui tujuan awalnya, dan pada akhir 1990-an sudah menjadi pilihan dominan untuk pengembangan sisi *server* [1]. Kemudahan penggunaan, dukungan *hosting* yang luas, dan ekosistem pustaka yang terus bertumbuh menjadikannya pilihan yang wajar bagi banyak pengembang web, mulai dari individu hingga tim teknologi di institusi besar [1].

Data terbaru menegaskan dominasi ini. Menurut W3Techs per 8 April 2026, sebesar 71,7% dari seluruh situs web yang memiliki bahasa pemrograman sisi *server* yang diketahui menggunakan PHP [2]. Angka itu menempatkan PHP jauh di atas bahasa-bahasa lainnya. Namun di balik angka yang terlihat impresif tersebut ada kenyataan yang kurang nyaman: hanya 58,1% dari situs web berbasis PHP yang telah beralih ke versi 8. Sekitar 33,1% masih menggunakan PHP 7.x, dan 8,7% masih berjalan di PHP 5.x [2]. Artinya hampir separuh pengguna PHP di seluruh dunia masih menjalankan versi yang sudah tidak mendapatkan pembaruan keamanan resmi.

Ini bukan masalah kecil. Dukungan resmi untuk PHP 7.4, versi terakhir dalam seri 7.x, berakhir pada 28 November 2022 [3]. Setelah tanggal itu, tidak ada lagi pembaruan keamanan yang dirilis, termasuk untuk kerentanan baru yang terus ditemukan. Basis Data Kerentanan Nasional (NVD) yang dikelola NIST mencatat bahwa sistem yang berjalan di atas versi *end-of-life* secara efektif membiarkan celah keamanan yang sudah diketahui publik tetap terbuka tanpa kemungkinan ditambal [4]. Semakin lama sistem dibiarkan berjalan di versi lama, semakin besar akumulasi risiko yang ditanggung.

Jenis kerentanan yang paling umum ditemukan pada aplikasi PHP lama sudah terdokumentasi cukup baik. OWASP Top 10:2021 [5] mencantumkan injeksi sebagai salah satu ancaman paling serius pada aplikasi web. Pada kode PHP lama, injeksi SQL hampir selalu muncul dari penggunaan fungsi `mysql_query()` di mana *input* pengguna langsung digabungkan ke dalam pernyataan SQL tanpa parameterisasi. Kerentanan XSS (*cross-site scripting*) juga lazim karena banyak aplikasi PHP lama tidak menerapkan encode *output* secara konsisten. Selain itu masih ada masalah kriptografi lemah seperti penggunaan `md5()` untuk menyimpan kata sandi, penanganan sesi yang tidak aman, dan berbagai fungsi yang sudah dihapus dari PHP 8.x karena alasan keamanan [5]. Bahwa masalah-masalah ini sudah ada sejak lama bukan berarti sudah terselesaikan; Merlo,

Letarte, dan Antoniol [6] sudah mengangkat persoalan perlindungan otomatis aplikasi PHP dari injeksi SQL sejak 2007, dan solusi komprehensif masih belum tersedia hingga sekarang.

Situasi ini bukan hanya dialami oleh organisasi di luar negeri. DTI UGM mengelola sejumlah aplikasi web berbasis PHP yang sebagian di antaranya masih berjalan di atas versi lama, sehingga proses modernisasi menjadi kebutuhan yang mendesak. Kondisi ini merepresentasikan tantangan yang umum dihadapi institusi pendidikan dan pemerintahan yang membangun sistem informasi berbasis PHP pada era sebelum standar pengembangan modern diadopsi secara luas, di mana migrasi sering tertunda karena kompleksitas teknis dan kekhawatiran terhadap gangguan layanan yang sudah berjalan [3].

Secara teknis, migrasi dari PHP 7.x ke PHP 8.x bukan perkara sederhana. PHP 8.x membawa sejumlah *breaking changes* yang membuat kode PHP 7.x tidak bisa langsung dijalankan tanpa modifikasi [3]. Fungsi-fungsi yang sebelumnya umum digunakan seperti `create_function()`, `each()`, dan seluruh ekstensi `ext/mysql` dihapus. Sistem tipe menjadi jauh lebih ketat dengan tipe *union* dan tipe campuran, yang bisa memicu kesalahan *runtime* pada kode yang sebelumnya berjalan normal. PHP 8.x juga memperkenalkan fitur baru seperti argumen bernama, *match expression*, operator *nullsafe* (`?->`), dan kompilasi JIT [3]. Penanganan kesalahan pun berubah: sejumlah fungsi yang dulu mengembalikan `false` ketika gagal kini melempar pengecualian.

Melakukan semua perubahan ini secara manual pada ratusan atau ribuan *file* kode adalah pendekatan yang tidak realistis. Beberapa organisasi memilih pendekatan bertahap, beralih ke PHP 7.4 dahulu sebelum pindah ke PHP 8.x [3], yang menjadikan dukungan versi target yang dapat dikonfigurasi sebagai kebutuhan nyata, bukan sekadar fitur tambahan.

Alat otomatis yang tersedia saat ini sudah membantu, tapi masing-masing punya batas. Rector [7] adalah alat transformasi berbasis AST yang terbukti andal untuk perubahan sintaksis skala industri, sebagaimana ditunjukkan oleh Chen et al. [8] dalam studi migrasi kode TypeScript di Microsoft. Namun Rector tidak bisa menangani kasus yang membutuhkan pemahaman semantik. Penggantian `mysql_*()` dengan PDO atau MySQLi, misalnya, bukan sekadar mengganti nama fungsi; ini penulisan ulang logika akses basis data secara keseluruhan yang hanya bisa dilakukan jika konteks penggunaannya dipahami.

Di sisi keamanan, Semgrep [9] bisa mengidentifikasi pola kerentanan secara otomatis, dan PHPStan [10] bisa memeriksa kompatibilitas tipe kode dengan versi PHP target. Tapi tidak ada satu pun dari alat-alat ini yang secara inheren menggabungkan deteksi kerentanan, transformasi kode, dan kepatuhan standar keamanan dalam satu alur kerja yang terintegrasi. Akibatnya tim pengembang yang mengerjakan migrasi dan tim kea-

manan yang melakukan audit bekerja secara terpisah. Strobl et al. [11] menemukan dari tiga kasus transformasi kode skala besar di sektor pemerintahan bahwa ketiadaan mekanisme validasi yang melekat pada proses transformasi menimbulkan risiko yang baru terasa jauh setelah sistem berjalan di produksi.

Tekanan bertambah bagi organisasi yang beroperasi di bawah standar keamanan informasi. ISO/IEC 27001:2022 [12] mensyaratkan praktik *secure coding* yang terdokumentasi dan dapat diaudit melalui tiga kontrol dalam Annex A.8: A.8.25 (*secure development lifecycle*), A.8.28 (*secure coding*), dan A.8.29 (pengujian keamanan dalam pengembangan dan penerimaan). Bagi organisasi yang sedang dalam proses sertifikasi ISO/IEC 27001:2022, bukti bahwa migrasi kode dilakukan dengan memperhatikan standar ini harus tersedia sebagai bagian dari dokumentasi audit. Menghasilkan bukti tersebut saat ini masih memerlukan pekerjaan manual yang terpisah dari proses migrasi.

Kemajuan dalam *Large Language Models* (LLM) membuka kemungkinan baru. LLM sudah terbukti dapat digunakan untuk mendeteksi kerentanan perangkat lunak [13] [14]. Li, Dutta, dan Naik [15] menunjukkan bahwa menggabungkan LLM dengan alat analisis statis menghasilkan deteksi kerentanan yang lebih baik dari masing-masing pendekatan yang berjalan sendiri. Stümpfle et al. [16] meneliti penggunaan LLM untuk transformasi kode yang membutuhkan pemahaman semantik dalam konteks migrasi varian perangkat lunak. Dengan kemampuan memahami konteks kode, LLM berpotensi mengisi celah yang tidak bisa ditangani alat berbasis aturan seperti Rector.

Kendala utamanya adalah privasi data. Model LLM berperforma tinggi seperti GPT-4 diakses melalui API *cloud*, yang berarti kode sumber harus dikirimkan ke *server* pihak ketiga. Untuk institusi seperti DTI UGM yang mengelola kode sumber aplikasi internal yang bersifat rahasia, ini bukan pilihan yang bisa diterima. Perkembangan model LLM *open source* berukuran kecil hingga menengah beberapa tahun terakhir mengubah situasi ini. Model-model seperti DeepSeek Coder 6.7b [17], Qwen2.5-Coder 7b [18], CodeLlama 7b [19], Mistral 7b [20], dan Llama 3.1 8b [21] kini bisa dijalankan secara lokal pada perangkat keras konsumen ber-VRAM 8 GB melalui Ollama [22]. Seluruh proses inferensi terjadi di mesin lokal tanpa ada data yang keluar dari lingkungan organisasi.

Dari uraian di atas, ada dua masalah mendasar yang belum terpecahkan sekaligus. Pertama, migrasi kode PHP lama masih memerlukan intervensi manual besar untuk kasus yang butuh pemahaman semantik, dan alat yang ada tidak menyertakan pemeriksaan keamanan sebagai bagian dari alur migrasi. Kedua, organisasi yang harus memenuhi ISO/IEC 27001:2022 masih harus melakukan audit keamanan sebagai langkah terpisah setelah migrasi selesai. Belum ada platform yang menggabungkan keduanya dalam satu alur kerja *open source* yang dapat dijalankan secara lokal, sebagaimana ditunjukkan oleh tinjauan literatur pada Bab II.

Penelitian ini membangun *platform* berbasis AI dan alat *open source* yang dapat secara otomatis mengubah aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi, dengan dukungan teknis dari PHP 5.2 hingga PHP 8.x, sekaligus menyertakan pemeriksaan *secure coding* berbasis ISO/IEC 27001:2022. *Platform* ini menggunakan Rector [7] untuk transformasi kode berbasis AST, Semgrep [9] untuk pemindaian keamanan, PHPStan [10] untuk validasi pasca-konversi, serta lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [22]. Studi kasus dilakukan pada kode PHP milik DTI UGM.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini merumuskan tiga permasalahan sebagai berikut:

1. Bagaimana merancang sistem berbasis AI yang mampu melakukan konversi otomatis aplikasi web PHP lama ke versi PHP tertentu sambil mempertahankan fitur aplikasi?
2. Bagaimana mengintegrasikan pemeriksaan *secure coding* yang selaras dengan standar ISO/IEC 27001:2022 ke dalam proses konversi otomatis?
3. Bagaimana memvalidasi bahwa hasil konversi memenuhi persyaratan fungsional dan keamanan ISO/IEC 27001:2022?

1.3 Tujuan Penelitian

Penelitian ini memiliki empat tujuan yang menjawab rumusan masalah di atas:

1. Mengembangkan *platform* berbasis AI yang menggunakan alat *open source* untuk secara otomatis mengubah aplikasi web PHP lama ke versi PHP target yang dapat dikonfigurasi tanpa kehilangan fungsionalitasnya.
2. Mengimplementasikan mekanisme pemeriksaan *secure coding* yang terhubung dengan kontrol dalam ISO/IEC 27001:2022 Annex A.8 [12], yang akan menjaga keamanan kode yang dikonversi.
3. Membangun sistem *quality assurance* yang memeriksa hasil konversi terhadap kebenaran fungsional dan standar kepatuhan keamanan yang ditetapkan oleh ISO/IEC 27001:2022 [12].
4. Menghasilkan dokumentasi otomatis berupa laporan konversi dan pemetaan kepatuhan terhadap ISO/IEC 27001:2022 [12], untuk membantu audit keamanan informasi.

1.4 Batasan Penelitian

Penelitian ini memiliki batasan-batasan sebagai berikut:

1. **Ruang lingkup konversi:** Secara teknis, *pipeline* yang dibangun mendukung konversi dari PHP 5.2 ke atas, karena Rector menggunakan rantai *ruleset* kumulatif yang mencakup semua perubahan dari PHP 5.2 hingga versi target yang dipilih [7]. Namun studi kasus dalam penelitian ini menggunakan kode sumber PHP 7.x milik DTI UGM sebagai representasi kondisi yang paling umum ditemukan saat ini [2]. Versi target yang dapat dikonfigurasi mencakup PHP 7.4 hingga PHP 8.3, yang merupakan rentang versi paling relevan untuk kebutuhan modernisasi institusi saat ini [3].
2. **Studi kasus:** Fokus utama studi kasus ini adalah aplikasi web PHP milik Direktorat Teknologi Informasi (DTI) di Universitas Gadjah Mada, yang dibangun di atas *framework* CodeIgniter 3.x. *Pipeline* yang dikembangkan bekerja pada level *file* PHP individual dan tidak bergantung pada *framework* yang digunakan, sehingga secara teknis dapat diterapkan pada aplikasi berbasis *framework* lain seperti Laravel atau Symfony maupun pada aplikasi PHP tanpa *framework*. Pengujian terhadap *framework* selain CodeIgniter 3.x bukan bagian dari ruang lingkup penelitian ini.
3. **Kontrol keamanan:** Pemeriksaan keamanan hanya dilakukan pada kontrol ISO/IEC 27001:2022 Annex A yang relevan dengan *secure coding*: A.8.25, A.8.28, dan A.8.29, serta A.5.17 dan A.8.24 dan A.8.26 yang turut dipetakan oleh *pipeline* [12].
4. **Model AI:** Penelitian ini mengevaluasi dan membandingkan lima model LLM *open source* yang dijalankan secara lokal melalui Ollama [22] dalam eksperimen komparasi *zero-shot*: DeepSeek Coder 6.7b [17], Qwen2.5-Coder 7b [18], CodeLlama 7b [19], Mistral 7b [20], dan Llama 3.1 8b [21]. Semua model digunakan tanpa *fine-tuning*.
5. **Lingkungan validasi:** Pengujian dilakukan dalam pengaturan lokal menggunakan kode sumber dari lingkungan institusional. Pengujian dalam lingkungan produksi skala besar bukan bagian dari ruang lingkup penelitian ini.
6. **Bahasa pemrograman:** Penelitian ini terbatas pada aplikasi PHP. Bahasa lain tidak dicakup.
7. **Kompleksitas aplikasi:** Aplikasi uji memiliki kompleksitas tingkat menengah, dengan maksimum 10.000 baris kode.

1.5 Manfaat Penelitian

1.5.1 Manfaat Akademis

Penelitian ini memberikan contoh nyata tentang cara mengintegrasikan kontrol *secure coding* ISO/IEC 27001:2022 ke dalam alur kerja transformasi kode otomatis [12], sesuatu yang belum dilakukan oleh penelitian-penelitian sebelumnya di bidang ini [6] [13] [16]. Selain itu, penelitian ini menghasilkan data empiris tentang performa model-model LLM *open source* lokal berukuran kecil hingga menengah dalam tugas analisis keamanan kode, yang relevan bagi institusi dengan sumber daya komputasi terbatas.

Desain modular *platform* memungkinkan pengembangan lebih lanjut oleh peneliti lain, misalnya dengan mengganti alat transformasi untuk mendukung bahasa pemrograman lain, menggunakan model AI yang lebih baru, atau memperluas pemetaan ke standar keamanan lain selain ISO/IEC 27001:2022.

1.5.2 Manfaat Praktis

Platform yang dibangun memudahkan pemindahan aplikasi PHP lama, khususnya bagi organisasi yang tidak memiliki banyak sumber daya pengembangan. Pemeriksaan kepatuhan keamanan disertakan langsung dalam proses migrasi [12], sehingga kerentanan dapat ditemukan bersamaan dengan konversi dan laporan kepatuhan dihasilkan secara otomatis. Dukungan versi target dari PHP 5.2 hingga PHP 8.x mengakomodasi berbagai strategi migrasi, baik yang bertahap maupun yang langsung. Seluruh *platform* bersifat *open source* dan dapat digunakan secara gratis.

1.6 Sistematika Penulisan

Skripsi ini terdiri dari lima bab.

Bab I: Pendahuluan memaparkan latar belakang masalah, rumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan.

Bab II: Tinjauan Pustaka dan Landasan Teoritis membahas penelitian-penelitian sebelumnya yang relevan dan kerangka teoritis, mencakup perubahan versi PHP, standar ISO/IEC 27001:2022, metodologi *secure coding*, analisis kode berbasis LLM, serta alat-alat yang digunakan: Rector, Semgrep, PHPStan, dan Ollama.

Bab III: Metodologi Penelitian menjelaskan arsitektur sistem, tahapan penelitian, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, serta metode pengujian dan evaluasi.

Bab IV: Hasil dan Diskusi menyajikan hasil implementasi pada studi kasus DTI UGM, mencakup tingkat keberhasilan konversi, perbandingan temuan keamanan sebelum dan sesudah konversi, hasil komparasi lima model LLM, dan diskusi mengenai kekuatan dan

keterbatasan sistem.

Bab V: Kesimpulan dan Rekomendasi merangkum temuan berdasarkan tujuan yang ditetapkan dan memberikan saran untuk penelitian berikutnya.

BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Literatur

Bagian ini membahas penelitian-penelitian sebelumnya yang relevan dengan tiga fokus utama studi ini: analisis keamanan otomatis pada aplikasi PHP, deteksi kerentanan berbasis LLM, dan transformasi kode yang melibatkan LLM. Masing-masing karya dibahas dari sisi masalah yang diatasi, kontribusinya, dan keterbatasannya. Bagian ini ditutup dengan tabel perbandingan yang memperlihatkan posisi penelitian ini di antara karya-karya tersebut.

2.1.1 Perlindungan Otomatis Aplikasi PHP dari Injeksi SQL

Merlo, Letarte, dan Antoniol [6] pada 2007 sudah menunjukkan bahwa analisis keamanan dan perbaikan kode PHP dapat diotomatisasi. Pendekatan mereka menggabungkan analisis statis, analisis dinamis, dan rekayasa ulang kode untuk melindungi aplikasi PHP lama dari injeksi SQL tanpa mengubah fungsionalitasnya. Caranya adalah dengan mengekstrak penjaga berbasis model dari kasus uji tepercaya, lalu secara otomatis menyisipkannya ke titik-titik akses basis data dalam kode sumber.

Hasilnya cukup kuat: pengujian pada phpBB versi 2.0.0 (37.193 baris kode PHP 4.2.2) berhasil menghentikan semua 176 serangan injeksi yang diuji tanpa satu pun *false positive*. Tapi ada batasnya. Kualitas perlindungan sepenuhnya bergantung pada kelengkapan rangkaian pengujian yang digunakan selama fase analisis dinamis. Titik akses yang tidak tercakup pengujian tidak terlindungi.

Yang relevan dari karya ini untuk penelitian saat ini bukan teknisnya, tapi konfirmasinya: otomatisasi analisis keamanan dan remediasi kode PHP dalam skala besar memang bisa dilakukan. Fakta bahwa studi ini sudah muncul hampir dua dekade lalu, namun solusi komprehensif yang bekerja dengan standar keamanan modern masih belum ada, justru mempertegas kesenjangan yang ingin diisi oleh penelitian ini.

2.1.2 Menggunakan LLM untuk Menemukan dan Memantau Kerentanan

Akuthota dkk. [13] meneliti kemampuan GPT-3.5-Turbo untuk deteksi dan pemantauan kerentanan perangkat lunak. Pendekatannya murni berbasis *prompt engineering* tanpa pelatihan ulang model sama sekali. Model berhasil menemukan pola kerentanan yang umum, meskipun akurasinya menurun seiring meningkatnya kompleksitas pola yang dicari.

Li et al. [14] mengambil pendekatan yang lebih terstruktur. Mereka menggabungkan analisis *patch* historis dengan analisis statis, lalu menambahkan tabel fitur ke-

rentanan dan templat penganalisis untuk membuat klasifikasi LLM lebih akurat, terutama pada kerentanan yang membutuhkan pemahaman aliran data lintas fungsi. Evaluasi pada *dataset* CVE kernel Linux memperlihatkan hasil yang lebih baik dibandingkan analisis statis saja.

Dua temuan penting bisa ditarik dari kedua studi ini. LLM tujuan umum sudah bisa menemukan pola kerentanan hanya dengan *prompt engineering*, dan penggabungan LLM dengan analisis statis terstruktur memberikan hasil yang lebih akurat untuk kasus yang kompleks. Masalah yang sama muncul di keduanya: model dijalankan via API *cloud*, yang jadi hambatan nyata ketika kode yang dianalisis bersifat rahasia. Penelitian ini menyelesaikan kendala itu dengan menjalankan model secara lokal melalui Ollama, tanpa ada kode sumber yang keluar dari lingkungan organisasi.

2.1.3 Transformasi Kode dengan Bantuan LLM

Stümpfle dkk. [16] mengusulkan pendekatan untuk mengubah varian perangkat lunak yang dikloning menjadi *Software Product Line* (SPL) menggunakan LLM. Metodenya bekerja dalam tiga tahap: penyaringan titik variasi untuk menemukan perbedaan kode yang relevan secara semantik antar varian, rekayasa *prompt* untuk menghasilkan artefak SPL yang benar, dan *loop* umpan balik penyempurnaan diri yang mengirimkan hasil pengujian kembali ke model secara berulang hingga kode lolos kompilasi dan pengujian fungsional. *Loop* umpan balik ini terbukti secara signifikan mengurangi kegagalan akibat halusinasi model, meskipun efektivitasnya tetap bergantung pada kualitas langkah penyaringan awal.

Relevansinya dengan penelitian ini cukup langsung. Keduanya menggunakan LLM untuk transformasi kode yang melampaui substitusi sintaksis, yang butuh pemahaman semantik. *Loop* umpan balik dalam Stümpfle dkk. memiliki fungsi konseptual yang mirip dengan peran validasi PHPStan dalam *pipeline* penelitian ini: keduanya menggunakan *output* alat analisis statis untuk memverifikasi apakah transformasi berhasil. Perbedaannya terletak pada fokus: Stümpfle et al. menangani ekstraksi fitur lintas varian untuk rekayasa lini produk, sedangkan penelitian ini berfokus pada migrasi versi dan verifikasi kepatuhan keamanan untuk kode warisan dalam satu bahasa.

2.1.4 Pendekatan Neuro-Simbolik untuk Deteksi Kerentanan Keamanan

Li, Dutta, dan Naik [15] memperkenalkan IRIS, sistem yang menggabungkan LLM dengan analisis statis untuk deteksi kerentanan pada tingkat seluruh repositori. Masalah yang diangkat cukup spesifik: alat analisis statis tradisional seperti CodeQL sangat bergantung pada spesifikasi *taint* yang dibuat manual, dan spesifikasi ini hampir pasti tidak lengkap karena jumlah API pihak ketiga terus bertambah. LLM juga tidak bisa berjalan sendiri karena tidak mampu melakukan penalaran yang mencakup keseluruhan

aliran kode dalam proyek berskala besar.

Solusi IRIS adalah membiarkan LLM menghasilkan spesifikasi *taint* secara otomatis berdasarkan pemahaman kontekstual terhadap API yang relevan, lalu menyerahkan analisis jalur ke alat analisis statis yang sudah ada. Pada *dataset* CWE-Bench-Java yang terdiri dari 120 kerentanan yang divalidasi secara manual, IRIS dengan GPT-4 berhasil mendeteksi 55 kerentanan dibandingkan hanya 27 oleh CodeQL saja, sekaligus menurunkan rata-rata *false positive* sebesar 5,21 poin persentase.

Ini adalah bukti empiris langsung bahwa kombinasi LLM dan analisis statis lebih baik dari masing-masing yang berjalan sendiri, sesuatu yang menjadi dasar desain *pipeline* penelitian ini. Bedanya: IRIS mengandalkan GPT-4 via *cloud* dan hanya diuji pada Java. Penelitian ini mengambil pendekatan konseptual yang serupa tetapi dengan model lokal melalui Ollama dan menargetkan PHP.

2.1.5 Dampak Jangka Panjang Transformasi Kode Otomatis

Strobl dkk. [11] menganalisis tiga kasus transformasi kode otomatis skala besar di sebuah organisasi pemerintah Austria, semuanya melibatkan konversi sistem warisan dari COBOL dan PL/I ke Java dengan ukuran lebih dari satu juta baris kode. Yang membedakan studi ini dari yang lain adalah fokusnya bukan pada keberhasilan transformasi itu sendiri, melainkan pada apa yang terjadi setelahnya.

Hasilnya mengganggu. Transformasi otomatis seringkali menghasilkan kode yang sintaksis valid di bahasa target, tapi mempertahankan karakteristik struktural dari bahasa sumber sehingga sulit dipahami pengembang yang tidak kenal sistem lama. Lebih kritis lagi: kasus yang memiliki *test suite* otomatis sejak awal jauh lebih mudah dirawat pascatransformasi. Kasus tanpa pengujian otomatis menjadi beban jangka panjang.

Temuan ini secara langsung memperkuat keputusan desain dalam penelitian ini untuk menyertakan PHPStan sebagai validasi pasca-konversi dan Semgrep sebagai pemindaian keamanan sebelum dan sesudah transformasi. Masalah yang ditinggalkan oleh konversi otomatis perlu terdeteksi segera, bukan bertahun-tahun kemudian.

2.1.6 Transformasi Kode Berbasis Pola untuk Migrasi Aplikasi

Cai et al. [23] mengusulkan pendekatan transformasi kode berbasis pola untuk migrasi aplikasi ke lingkungan *cloud*. Prosesnya iteratif: pemindaian kode untuk mengidentifikasi blok yang perlu diubah, pencocokan pola menggunakan ekspresi reguler yang diperluas, perancangan templat untuk kode target, lalu eksekusi aturan transformasi. Pendekatan ini diujian pada 19 proyek *open source* Java dengan memigrasikannya ke Amazon Web Services (AWS).

Kontribusinya jelas: transformasi berbasis pola terbukti bisa mengurangi pekerja-

an manual berulang pada migrasi skala besar. Tapi ada keterbatasan mendasar yang tidak bisa dihindari. Ekspresi reguler yang diperluas tetap beroperasi pada level teks, bukan pada representasi struktural kode. Ini membuatnya rentan gagal pada konstruksi kode yang kompleks atau tidak mengikuti pola yang telah didefinisikan sebelumnya.

Perbandingan dengan pendekatan yang digunakan dalam penelitian ini cukup informatif. Rector [7] bekerja pada AST, bukan teks, sehingga transformasinya lebih dapat diandalkan untuk kode produksi yang beragam. Selain itu, Cai et al. [23] tidak menyertakan aspek keamanan atau kepatuhan standar sama sekali dalam pendekatan mereka. Perbedaan ini mencerminkan bagaimana kebutuhan dalam transformasi kode otomatis telah berkembang: dari sekadar memindahkan kode ke lingkungan baru, menjadi memastikan kode yang dipindahkan juga aman.

2.1.7 Ringkasan Perbandingan

Tabel 2.1 dan Tabel 2.2 merangkum ketujuh studi yang dibahas di atas.

Tabel 2.1. Perbandingan Penelitian Terdahulu dan Karakteristik Penelitian

Studi	Bahasa	AI/LLM Digu- nakan	Tugas Utama
Merlo et al. [6]	PHP	Tidak ada	<i>Security remediation</i>
Akuthota et al. [13]	<i>Multiple</i>	GPT-3.5-Turbo	<i>Vulnerability detection</i>
Li et al. [14]	C (Linux kernel)	LLM + analisis statis	<i>Vulnerability detection</i>
Stümpfle et al. [16]	C/C++	LLM	<i>Code transformation</i>
Li, Dutta & Naik [15]	Java	GPT-4	<i>Vulnerability detection</i>
Strobl et al. [11]	COBOL/PL/I	Tidak ada	<i>Code transformation</i>
Cai et al. [23]	Java	Tidak ada	<i>Cloud migration</i>
Penelitian ini	PHP	5 model LLM lokal	Migrasi versi + <i>security compliance</i>

Kesenjangan yang konsisten terlihat dari kedua tabel. Studi yang berfokus pada keamanan tidak menangani migrasi versi. Studi yang berfokus pada transformasi kode tidak menyertakan pemeriksaan keamanan maupun pemetaan ke standar formal. Tidak satu pun dari ketujuh studi yang menggabungkan migrasi versi PHP, analisis keamanan berbasis LLM, validasi pasca-konversi, dan kepatuhan ISO/IEC 27001:2022 dalam satu alur kerja *open source* dengan *deployment* lokal. Itulah yang ingin dicapai penelitian ini.

Tabel 2.2. Perbandingan Penelitian Terdahulu dan Aspek Metodologi

Studi	<i>Deployment</i>	Validasi Pasca-Konversi	Standar Keamanan	<i>Open Source</i>
Merlo et al. [6]	N/A	Tidak	Tidak ada	Ya
Akuthota et al. [13]	<i>Cloud</i>	Tidak	Tidak ada	Tidak
Li et al. [14]	<i>Cloud</i>	Tidak	Tidak ada	Tidak
Stümpfle et al. [16]	<i>Cloud</i>	Ya (<i>feedback loop</i>)	Tidak ada	Tidak
Li, Dutta & Naik [15]	<i>Cloud</i>	Tidak	Tidak ada	Ya (IRIS)
Strobl et al. [11]	N/A	Tidak	Tidak ada	Tidak
Cai et al. [23]	N/A	Tidak	Tidak ada	Tidak
Penelitian ini	Lokal (Ollama)	Ya (PHPStan + Semgrep)	ISO/IEC 27001:2022	Ya

2.2 Dasar Teori

2.2.1 PHP dan Perubahan Antar Versi

PHP (*Hypertext Preprocessor*) menjadi bahasa skrip sisi *server* yang paling banyak digunakan di web sejak akhir 1990-an [1]. W3Techs mencatat bahwa 71,7% dari seluruh situs web yang memiliki bahasa pemrograman sisi *server* menggunakan PHP [2].

Untuk memahami tantangan migrasi yang dihadapi kode lama, perlu dipahami dulu karakteristik PHP 5.x yang masih banyak ditemukan di sistem yang belum bermigrasi. PHP 5.x, aktif digunakan dari sekitar 2004 hingga awal 2010-an, memiliki karakteristik yang berbeda secara fundamental dari versi modern. Yang paling signifikan adalah ekstensi `ext/mysql` dengan fungsi-fungsi seperti `mysql_connect()`, `mysql_query()`, dan `mysql_fetch_array()`. Fungsi-fungsi ini tidak mendukung *prepared statements* secara asli, sehingga rentan terhadap injeksi SQL jika *input* pengguna tidak disanitasi secara manual. Ekstensi ini sudah dinyatakan *deprecated* sejak PHP 5.5 dan dihapus sepenuhnya di PHP 7.0 [3]. PHP 5.x juga menggunakan kata kunci `var` untuk deklarasi properti kelas, sintaks `array()` yang panjang, dan belum mendukung *type hints* skalar. Rector dapat menangani sebagian besar transformasi sintaksis dari PHP 5.x melalui *ruleset* kumulatifnya [7], tapi penggantian `mysql_*` tidak bisa dilakukan Rector karena butuh pemahaman semantik tentang konteks penggunaan basis data. Keterbatasan inilah yang secara langsung memotivasi kehadiran AI Engine dalam *pipeline* penelitian ini.

PHP 7.x membawa peningkatan performa signifikan lewat Zend Engine 3.0, menambahkan deklarasi tipe skalar, petunjuk tipe pengembalian, dan operator penggabungan *null*. PHP 7.4, versi terakhir seri ini, menambahkan properti bertipe dan fungsi panah

sebelum dukungan resminya berakhir 28 November 2022 [3].

PHP 8.x membuat sejumlah *breaking changes* yang membuat kode PHP 7.x tidak bisa langsung dijalankan [3]. Selain penghapusan fungsi-fungsi yang sudah *deprecated*, sistem tipe menjadi jauh lebih ketat dengan tipe *union* dan tipe campuran. Fitur baru seperti *match expression*, operator *nullsafe* (`?->`), argumen bernama, dan kompilasi JIT diperkenalkan. Penanganan kesalahan juga berubah: beberapa fungsi yang sebelumnya mengembalikan `false` kini melempar pengecualian. Untuk basis kode besar, melakukan semua perubahan ini secara manual jelas tidak realistis.

2.2.2 ISO/IEC 27001:2022 dan *Secure Coding*

ISO/IEC 27001:2022 adalah standar internasional untuk Sistem Manajemen Keamanan Informasi (ISMS) yang dikembangkan bersama oleh ISO dan IEC [12]. Standar ini menetapkan persyaratan untuk membuat, menerapkan, memelihara, dan terus meningkatkan ISMS dalam suatu organisasi. Annex A memuat 93 kontrol keamanan yang dibagi ke dalam empat domain: Organisasi, Manusia, Fisik, dan Teknologi. Dari 93 kontrol tersebut, penelitian ini hanya memetakan yang langsung relevan dengan pengembangan perangkat lunak:

- **A.8.25** (*Secure development lifecycle*) mengharuskan aturan pengembangan yang aman ditetapkan dan diikuti di setiap tahap, termasuk penggunaan pedoman *secure coding* dan alat analisis statis.
- **A.8.28** (*Secure coding*) mengharuskan aturan pengkodean yang aman diikuti untuk mencegah kerentanan umum seperti injeksi, autentikasi yang rusak, pengungkapan data sensitif, dan kriptografi yang tidak aman.
- **A.8.29** (Pengujian keamanan dalam pengembangan dan penerimaan) mengharuskan pengujian keamanan menjadi bagian dari proses pengembangan sebelum kode masuk produksi.

Ketiga kontrol ini, bersama A.5.17, A.8.24, dan A.8.26 yang turut dipetakan oleh *pipeline*, menjadi acuan resmi untuk mesin pengecekan keamanan dalam penelitian ini [12]. Aturan Semgrep yang digunakan berhubungan langsung dengan A.8.28 dan A.8.29. Struktur *pipeline* secara keseluruhan mencerminkan kebutuhan siklus hidup A.8.25.

2.2.3 OWASP Top 10 dan Kerentanan Aplikasi Web PHP

OWASP Top 10 adalah referensi standar untuk risiko keamanan paling kritis pada aplikasi web yang diterbitkan secara berkala oleh OWASP berdasarkan data dari ratusan organisasi di seluruh dunia [5]. Versi terbaru, OWASP Top 10:2021, relevan langsung dengan kondisi aplikasi PHP lama.

Injeksi menempati posisi ketiga dan merupakan yang paling umum ditemukan pada kode PHP lama [5]. Injeksi SQL hampir selalu berasal dari penggunaan `mysql_query()` tanpa parameterisasi. XSS masuk dalam kategori injeksi yang sama dan muncul ketika *output* tidak diencode sebelum ditampilkan ke peramban. Kegagalan kriptografi, yang menempati posisi kedua, kerap hadir dalam bentuk penggunaan `md5()` atau `sha1()` untuk menyimpan kata sandi, padahal keduanya sudah lama tidak dianggap aman untuk keperluan itu.

Dalam penelitian ini, Semgrep dijalankan dengan *ruleset* `p/php` dan `p/owasp-top-ten` [9], yang secara langsung dipetakan ke kategori-kategori dalam OWASP Top 10:2021. Temuan Semgrep kemudian dipetakan ke kontrol ISO/IEC 27001:2022 yang relevan melalui `iso_mapper.py`.

2.2.4 DevSecOps dan Integrasi Keamanan dalam Pipeline Pengembangan

DevSecOps adalah praktik yang mengintegrasikan keamanan langsung ke dalam setiap tahap siklus hidup pengembangan perangkat lunak [14]. Berbeda dari pendekatan tradisional yang menempatkan pengujian keamanan sebagai langkah terpisah di akhir proses, DevSecOps memperlakukan keamanan sebagai bagian dari setiap keputusan teknis selama pengembangan berlangsung, bukan tanggung jawab tim khusus yang bekerja setelah kode selesai ditulis.

Dalam praktiknya, DevSecOps diwujudkan melalui integrasi alat keamanan otomatis ke dalam *pipeline*: analisis statis kode sumber (SAST), pemindaian dependensi, dan pengujian keamanan dinamis. Kerentanan yang ditemukan lebih awal jauh lebih murah untuk diperbaiki dibandingkan yang baru ditemukan setelah sistem berjalan di produksi.

Pipeline dalam penelitian ini menerapkan prinsip DevSecOps secara langsung: Semgrep memindai sebelum dan sesudah konversi, PHPStan memvalidasi kompatibilitas kode, dan AI Engine menangani kasus yang butuh analisis semantik. Keamanan bukan langkah audit terpisah, tapi bagian dari proses migrasi itu sendiri [12].

2.2.5 Abstract Syntax Trees dan Static Analysis

Analisis statis adalah teknik pemeriksaan kode sumber tanpa menjalankan program. Dalam konteks keamanan perangkat lunak, ini digunakan untuk mengidentifikasi pola kerentanan, praktik pengkodean tidak aman, dan pelanggaran kebijakan secara otomatis. Analisis statis sudah menjadi bagian standar dari *pipeline* DevSecOps modern karena bisa dijalankan terus-menerus selama pengembangan tanpa beban tambahan yang signifikan [14].

Abstract Syntax Trees (AST) adalah representasi pohon dari kode sumber di mana

setiap *node* mewakili konstruksi sintaksis seperti ekspresi, pernyataan, atau deklarasi. Alat yang bekerja dengan AST bisa menganalisis dan mengubah kode secara struktural, berbeda dari pendekatan berbasis teks biasa yang digunakan oleh Cai et al. [23]. Rector menggunakan PHP AST untuk menemukan pola yang perlu diubah dan menghasilkan kode penggantinya. Chen et al. [8] mendemonstrasikan bahwa pendekatan berbasis AST bukan sekadar teori; mereka menerapkannya untuk migrasi kode TypeScript di Microsoft pada skala produksi.

2.2.6 Rector: Alat Transformasi Kode Berbasis AST

Rector adalah alat *open source* untuk transformasi kode PHP berbasis AST [7]. Prosesnya: kode sumber PHP diubah menjadi AST, aturan transformasi diterapkan, lalu kode PHP baru dihasilkan dari pohon yang sudah dimodifikasi. Rector dilengkapi aturan bawaan untuk berpindah antar versi PHP, termasuk `LevelSetList::UP_TO_PHP_80` hingga `UP_TO_PHP_83`, dan *ruleset*-nya bersifat kumulatif sehingga mencakup semua perubahan dari versi sumber hingga versi target.

Dalam penelitian ini, Rector berperan sebagai mesin transformasi utama. Transformasi sintaksis dan struktural dari PHP 5.x atau 7.x ke PHP 8.x ditangani sepenuhnya oleh Rector. Yang tidak bisa ditangani adalah penggantian `ext/mysql` karena itu butuh penulisan ulang semantik, bukan sekadar penggantian sintaks. Itulah yang ditangani oleh AI Engine.

2.2.7 PHPStan: Alat Analisis Statis untuk PHP

PHPStan adalah alat analisis statis untuk PHP yang memeriksa kebenaran tipe data, konsistensi pemanggilan fungsi, dan kesalahan logika tanpa menjalankan program [10]. Berbeda dari Semgrep yang mencari pola kerentanan keamanan, PHPStan fokus pada integritas kode dari sisi kompatibilitas dan tipe.

PHPStan bekerja dengan sistem level 0–9, di mana level lebih tinggi berarti pemeriksaan lebih ketat. Dalam *pipeline* penelitian ini, PHPStan digunakan pada level 5 sebagai komponen validasi pasca-konversi. Setelah Rector selesai, PHPStan memeriksa apakah kode hasil konversi kompatibel dengan versi PHP target dan bebas dari kesalahan tipe. Peran ini mendukung kontrol A.8.29 ISO/IEC 27001:2022 [12] dengan memastikan kode yang dihasilkan secara otomatis tidak mengandung kesalahan yang bisa memengaruhi keandalan sebelum masuk produksi.

2.2.8 Semgrep: Alat Analisis Statis Ringan untuk Keamanan

Semgrep adalah alat analisis statis ringan yang mendukung lebih dari 30 bahasa pemrograman, termasuk PHP [9]. Aturan keamanannya ditulis dalam format YAML dengan sintaks yang mirip dengan kode sumber yang dianalisis, sehingga pengguna bisa

membuat aturan sendiri tanpa perlu memahami detail AST bahasa target. Registri aturan publiknya mencakup berbagai kerentanan umum seperti injeksi SQL, XSS, kriptografi lemah, dan deserialisasi tidak aman.

Dalam penelitian ini, Semgrep digunakan dua kali: sebelum konversi untuk menetapkan *baseline* keamanan kode asli, dan sesudah konversi untuk memastikan tidak ada kerentanan baru yang muncul. Kedua pemindaian ini mendukung kontrol A.8.28 dan A.8.29 ISO/IEC 27001:2022 [12].

2.2.9 Model-Model LLM untuk Analisis Kode

LLM (*Large Language Model*) adalah model bahasa neural berbasis *transformer* yang dilatih pada data teks dan kode dalam jumlah besar. LLM yang dilatih pada kode bisa memahami semantik kode, mengidentifikasi pola kerentanan, menyarankan perbaikan, dan menghasilkan kode baru [13]. Kemampuan ini berbeda dari analisis berbasis aturan: LLM mempertimbangkan konteks dan tujuan kode, bukan hanya strukturnya.

Penelitian ini mengevaluasi lima model LLM *open source* yang dijalankan secara lokal. Kriteria pemilihan ada dua: model harus bisa dijalankan di GPU dengan VRAM 8 GB menggunakan kuantisasi Q4, dan harus memiliki kemampuan pemahaman kode yang terdokumentasi dalam literatur ilmiah.

DeepSeek Coder 6.7b [17] memiliki 6,7 miliar parameter dan dilatih pada 2 triliun token dengan komposisi 87% kode sumber dari lebih dari 80 bahasa pemrograman.

Qwen2.5-Coder 7b [18] adalah model kode dari Alibaba dengan 7 miliar parameter, dilatih pada 5,5 triliun token kode, dan menunjukkan kemampuan yang kuat pada penyelesaian kode dan perbaikan *bug*.

CodeLlama 7b [19] dikembangkan Meta AI berbasis Llama 2. Model ini mendukung konteks hingga 100 ribu token dan tersedia dalam varian *instruct* yang dioptimalkan untuk mengikuti instruksi.

Mistral 7b [20] adalah model tujuan umum dari Mistral AI yang menggunakan mekanisme *grouped-query attention* (GQA) dan *sliding window attention* (SWA), membuatnya efisien secara komputasi.

Llama 3.1 8b [21] adalah model 8 miliar parameter dari Meta AI yang dilatih pada lebih dari 15 triliun token multibahasa dengan peningkatan signifikan dalam kemampuan penalaran dibanding pendahulunya.

Li, Dutta, dan Naik [15] sudah menunjukkan bahwa kombinasi LLM dan analisis statis menghasilkan deteksi kerentanan yang lebih baik dari masing-masing yang berjalan sendiri. Perbandingan performa kelima model di atas dalam konteks analisis keamanan kode PHP akan dibahas secara empiris di Bab IV.

2.2.10 Ollama: Menerapkan LLM Secara Lokal

Ollama adalah *platform open source* untuk menjalankan LLM secara lokal di infrastruktur pengguna [22]. *Platform* ini mengkuantisasi model secara otomatis agar sesuai sumber daya perangkat keras yang tersedia dan menyediakan API REST yang kompatibel dengan format OpenAI, sehingga mudah diintegrasikan ke aplikasi Python.

Alasan memilih Ollama bukan sekadar teknis. Kode sumber yang dianalisis dalam penelitian ini bersifat rahasia milik institusi. Mengirimkannya ke API *cloud* bukan pilihan yang bisa diterima. Dengan Ollama, semua inferensi terjadi di mesin lokal tanpa data keluar dari lingkungan organisasi [22], sejalan dengan prinsip ISO/IEC 27001:2022 [12] dan merupakan persyaratan eksplisit dari DTI UGM.

BAB III

METODOLOGI PENELITIAN

Bab ini menjelaskan langkah-langkah yang diambil dalam penelitian ini untuk mencapai tujuan yang ditetapkan dalam Subbab 1.3. Cakupannya meliputi alat dan bahan yang digunakan, metode penelitian, arsitektur sistem, alur data *pipeline*, desain *prompt* LLM, pemetaan kontrol ISO/IEC 27001:2022, rancangan eksperimen komparasi model, dan metode evaluasi.

3.1 Alat dan Bahan Penelitian

3.1.1 Perangkat Keras

Seluruh proses pengembangan dan pengujian dalam penelitian ini dilakukan pada satu unit laptop HP Victus 16-r0xxx dengan spesifikasi sebagai berikut:

- Sistem operasi: Windows 11 Home Single Language 64-bit (Build 26200)
- Prosesor: Intel Core i7-13700HX, 24 inti logis, kecepatan dasar 2,1 GHz
- Memori: 16.384 MB RAM
- GPU diskrit: NVIDIA GeForce RTX 4060 dengan VRAM 8 GB
- GPU terintegrasi: Intel UHD Graphics
- Penyimpanan: NVMe SSD 1 TB

Alasan penggunaan spesifikasi ini bukan sekadar ketersediaan. Model LLM yang dijalankan secara lokal melalui Ollama membutuhkan VRAM yang cukup agar inferensi bisa berjalan sepenuhnya di GPU. RTX 4060 dengan VRAM 8 GB memungkinkan kelima model yang dievaluasi (masing-masing 6–8 miliar parameter) dijalankan pada kuantisasi Q4_0 tanpa harus memindahkan sebagian perhitungan ke RAM atau CPU. Spesifikasi ini juga merepresentasikan perangkat keras kelas menengah yang realistis dimiliki oleh institusi atau pengembang individual, sehingga hasil pengujian tidak hanya berlaku di lingkungan riset yang ideal.

3.1.2 Perangkat Lunak

- **Python 3.x**: Bahasa pemrograman utama untuk membangun *pipeline* yang menghubungkan semua modul. Dipilih karena kemudahannya memanggil perintah eksternal via *subprocess* dan ketersediaan pustaka `requests` untuk berkomunikasi dengan API Ollama.
- **Rector** [7]: Alat transformasi kode berbasis AST untuk PHP, dikonfigurasi secara di-

namis sesuai versi target yang dipilih.

- **Semgrep** [9]: Alat analisis statis untuk pemindaian kerentanan. Dijalankan dengan *ruleset* `p/php` dan `p/owasp-top-ten` [5].
- **PHPStan** [10]: Alat analisis statis untuk validasi tipe dan kompatibilitas kode PHP, dijalankan pada level 5.
- **Ollama** [22]: *Platform* untuk menjalankan LLM secara lokal, diakses via API REST di `localhost:11434`.
- **PHP 8.5.1**: Interpreter PHP untuk validasi sintaks menggunakan `php -l`.
- **Composer**: Manajer dependensi PHP untuk instalasi Rector dan PHPStan.
- **Visual Studio Code** dengan ekstensi Claude Code sebagai IDE utama selama pengembangan.

3.1.3 Model LLM yang Dievaluasi

Lima model LLM *open source* dievaluasi, semuanya dijalankan secara lokal melalui Ollama tanpa *fine-tuning*. Dua kriteria pemilihan diterapkan: model harus bisa dijalankan di GPU VRAM 8 GB pada kuantisasi Q4_0, dan kemampuan pemahaman kodenya harus terdokumentasi dalam literatur ilmiah.

- DeepSeek Coder 6.7b [17]: Model spesialis kode, dilatih pada 2 triliun token dengan 87% kode sumber.
- Qwen2.5-Coder 7b [18] : Model kode dari Alibaba, dilatih pada 5,5 triliun token kode.
- CodeLlama 7b [19]: Model berbasis Llama 2 dari Meta AI untuk tugas pemrograman.
- Mistral 7b [20]: Model tujuan umum dari Mistral AI dengan arsitektur *grouped-query attention*.
- Llama 3.1 8b [21]: Model dari seri Llama 3 yang dilatih pada lebih dari 15 triliun token multibahasa.

3.1.4 Bahan Penelitian

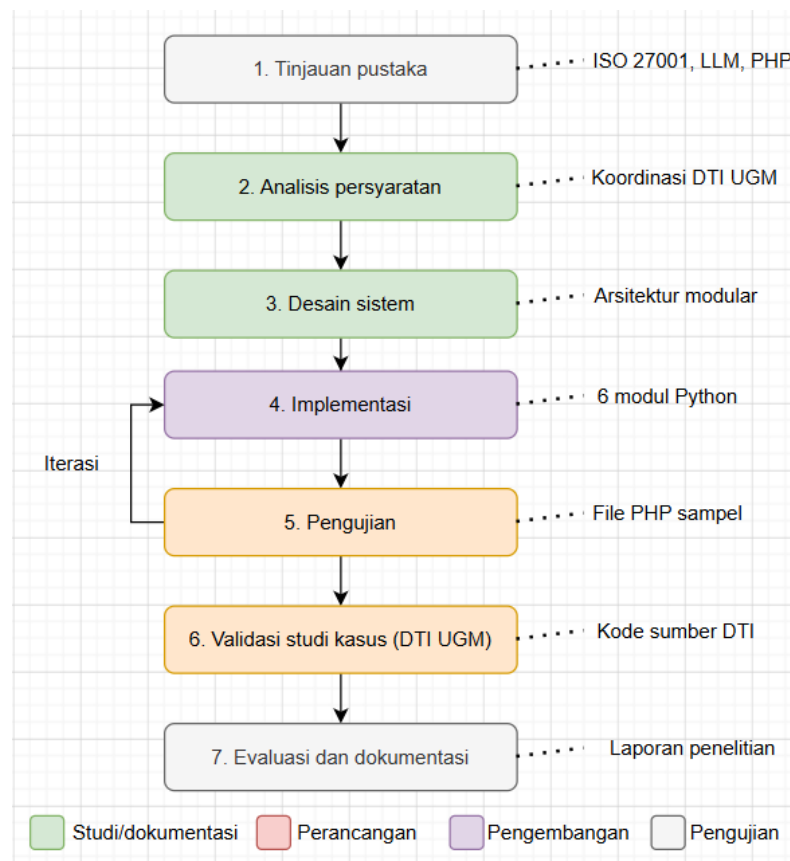
- **Dokumen ISO/IEC 27001:2022** [12]: Acuan utama untuk pemetaan kontrol keamanan.
- **Kode sumber PHP dari DTI UGM**: Kumpulan *file* PHP dari aplikasi web internal berbasis CodeIgniter 3.x. Bersifat rahasia dan hanya digunakan untuk analisis statis tanpa akses ke basis data produksi.
- **Dokumentasi resmi PHP** [3]: Referensi untuk memverifikasi kebenaran konversi Rector.

- **Registri aturan Semgrep** [9]: Aturan publik p/php dan p/owasp-top-ten untuk pemindaian keamanan.

3.2 Metode Penelitian

Penelitian ini menggunakan metodologi Penelitian dan Pengembangan (*Research and Development*, R&D) dengan model prototipe. Pilihan ini didasarkan pada fakta bahwa penelitian menghasilkan produk perangkat lunak nyata, bukan hanya analisis teoritis. Model prototipe dipilih karena dalam praktiknya banyak keputusan desain baru bisa dibuat setelah melihat perilaku nyata sistem, bukan dari perencanaan di atas kertas saja. Ini terutama berlaku untuk format *prompt* AI dan cara menangani kode yang tidak bisa diotomatisasi sepenuhnya.

Gambar 3.1 menunjukkan alur penelitian yang terdiri dari tujuh tahapan.



Gambar 3.1. Alur penelitian dengan model prototipe

1. **Tinjauan pustaka:** Mempelajari permasalahan migrasi PHP lama, persyaratan ISO-/IEC 27001:2022 [12], kemampuan model LLM *open source*, dan alat analisis statis yang relevan. Koordinasi awal dengan DTI UGM dilakukan pada tahap ini.
2. **Analisis persyaratan:** Mendefinisikan persyaratan fungsional dan non-fungsional berdasarkan hasil tinjauan pustaka dan koordinasi dengan DTI UGM, termasuk me-

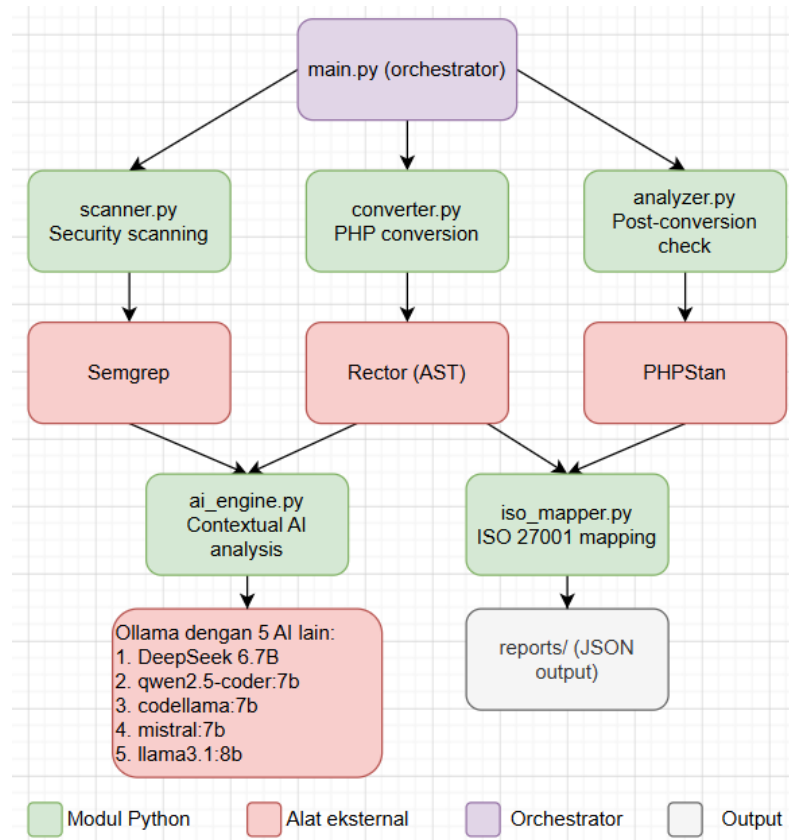
menentukan kontrol ISO yang dicakup dan persyaratan privasi data.

3. **Desain sistem:** Merancang arsitektur *pipeline* secara modular dan menentukan tanggung jawab setiap modul.
4. **Implementasi:** Membangun keenam modul Python satu per satu. Setiap modul diuji secara terpisah sebelum diintegrasikan.
5. **Pengujian:** Menguji prototipe dengan *file* PHP sampel. Jika ditemukan masalah, proses kembali ke tahap implementasi (iterasi).
6. **Validasi studi kasus:** Pengujian akhir menggunakan kode sumber asli DTI UGM setelah prototipe stabil.
7. **Evaluasi dan dokumentasi:** Menganalisis hasil dan menyusun laporan penelitian.

3.3 Arsitektur Sistem

Sistem terdiri dari enam modul Python yang masing-masing menangani satu langkah dalam proses konversi dan analisis keamanan. `main.py` bertindak sebagai orkes-trator yang mengontrol urutan eksekusi dan mengalirkan data antar modul. Arsitektur modular memungkinkan setiap bagian diuji, diperbaiki, atau diganti secara independen.

Gambar 3.2 menunjukkan hubungan antar modul.



Gambar 3.2. Arsitektur sistem *pipeline* dengan enam modul Python

3.3.1 Scanner (scanner.py)

Modul pertama yang berjalan. Scanner menjalankan Semgrep [9] pada direktori `input/` sebelum konversi dimulai untuk menetapkan *baseline* keamanan kode asli. Hasil pemindaian dikelompokkan berdasarkan jenis kerentanan (injeksi SQL, XSS, *path traversal*, injeksi perintah, fungsi usang, kriptografi lemah) dan diberi tingkat prioritas. Data ini menjadi pembanding untuk mengukur perubahan keamanan setelah konversi.

3.3.2 Converter (converter.py)

Modul ini menggunakan Rector [7] untuk transformasi kode PHP berbasis AST. Argumen CLI `-target-php` menentukan versi target; nilai yang diterima adalah "7.4", "8.0", "8.1", "8.2", atau "8.3". Sebelum Rector dijalankan, seluruh *file* PHP dari `input/` disalin dahulu ke `output/`. Rector kemudian memodifikasi *file* di `output/` di tempat. *File* yang gagal dikonversi tetap ada di `output/` sebagai salinan kode asli dan statusnya dicatat `FAILED` di laporan; direktori `input/` tidak pernah disentuh. Konfigurasi Rector dibuat sebagai *tempfile* saat *runtime* dan dihapus otomatis setelahnya.

Tabel 3.1. Pemetaan argumen `-target-php` ke konfigurasi Rector

Nilai <code>-target-php</code>	LevelSetList yang didefinisikan	Cakupan rule kumulatif
7.4	UP_TO_PHP_74	PHP 5.2, 5.3, 5.4, 5.5, 5.6, 7.0, 7.1, 7.2, 7.3, 7.4
8.0	UP_TO_PHP_80	Semua rule 7.4 + PHP 8.0
8.1	UP_TO_PHP_81	Semua rule 8.0 + PHP 8.1
8.2	UP_TO_PHP_82	Semua rule 8.1 + PHP 8.2
8.3	UP_TO_PHP_83	Semua rule 8.2 + PHP 8.3

Karena *ruleset*-nya kumulatif, kode PHP 5.x pun bisa diproses langsung menuju PHP 8.3 dalam satu langkah. Transformasi seperti `var` menjadi `public`, `array()` menjadi `[]`, dan penghapusan modifier `/e` pada `preg_replace()` ditangani sepenuhnya. Yang tidak bisa ditangani adalah `mysql_*()` karena butuh penulisan ulang semantik, bukan sekadar penggantian sintaks.

3.3.3 Analyzer (analyzer.py)

Setelah konversi, Analyzer menjalankan PHPStan [10] pada direktori `output/` untuk memverifikasi kompatibilitas kode dengan versi PHP target. Level 5 dipilih sebagai keseimbangan antara keketatan dan jumlah *false positive* yang wajar untuk kode warisan. Setiap temuan PHPStan dikategorikan sebagai `ERROR` atau `WARNING`, lalu dikaitkan dengan kontrol ISO/IEC 27001:2022 yang relevan.

3.3.4 AI Engine (*ai_engine.py*)

Modul ini menerima temuan dari Semgrep *pre-scan* sebagai masukan, bukan dari PHPStan dan bukan dari *post-scan*. Hanya temuan dengan prioritas 1–3 yang dikirim ke AI Engine: SQL Injection (prioritas 1), XSS (prioritas 2), dan fungsi usang (prioritas 3). Temuan prioritas 4 ke atas seperti *hardcoded credentials* dan *path traversal* tidak diproses.

Untuk setiap temuan yang lolos filter, modul mengirimkan konteks dan potongan kode ke model LLM melalui endpoint `/api/chat` Ollama [22] di `localhost:11434`. Kalau model tidak menghasilkan blok kode PHP sama sekali, respons tetap dicatat dengan *placeholder* `// Manual review required`, bukan di-skip. Jika Ollama tidak tersedia, *pipeline* tetap berjalan tanpa komponen AI. Desain *prompt* yang digunakan dibahas dalam Subbab 3.5.

3.3.5 ISO Mapper (*iso_mapper.py*)

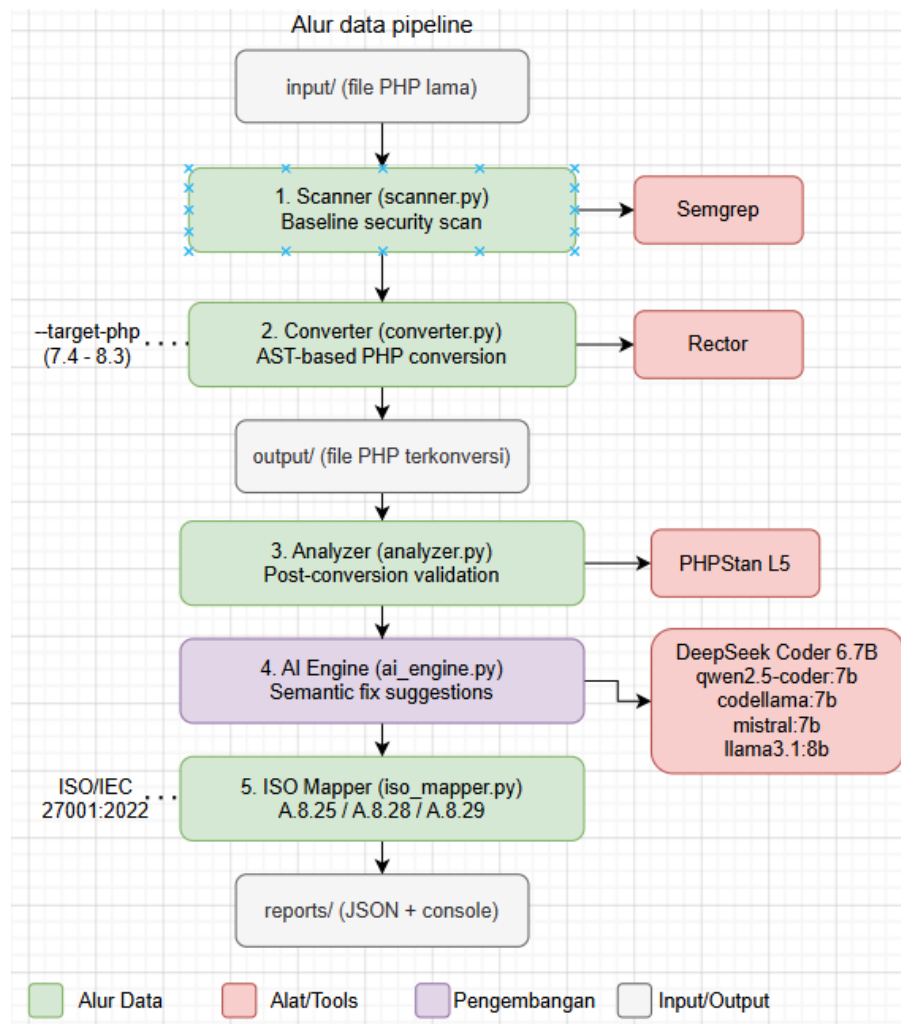
Modul ini menerima semua hasil dari Scanner, Analyzer, dan AI Engine sebagai objek Python *in-memory* langsung dari `main.py`, lalu memetakannya ke kontrol ISO/IEC 27001:2022 [12] secara berbasis aturan. Tidak ada pembacaan *file* JSON. Pemetaan berbasis aturan dipilih agar hasilnya deterministik dan dapat diaudit. Detail pemetaan dibahas dalam Subbab 3.6.

3.3.6 Main (*main.py*)

Orkestrator yang mengontrol seluruh *pipeline*. Menerima direktori *input* dan versi PHP target sebagai argumen CLI, menjalankan kelima modul secara berurutan, dan menghasilkan laporan JSON di direktori `reports/`. Ringkasan hasil ditampilkan di konsol menggunakan pustaka Rich.

3.4 Alur Data Pipeline

Gambar 3.3 menunjukkan alur data lengkap dari masukan hingga laporan akhir.



Gambar 3.3. Alur data *pipeline* dari *input* hingga laporan akhir

Alurnya cukup linear. *File* PHP ditempatkan di *input/*. Scanner memindai dengan Semgrep untuk menetapkan *baseline*. Converter menyalin semua *file* ke *output/* lalu menjalankan Rector dengan versi target yang ditentukan; *input/* tidak tersentuh. Analyzer menjalankan PHPStan level 5 pada *output/*. AI Engine mengambil temuan Semgrep prioritas 1–3 dan mengirimkannya ke model LLM. ISO Mapper mengumpulkan semua temuan dan memetakannya ke kontrol ISO. *main.py* menggabungkan semuanya menjadi laporan JSON dan menampilkan ringkasan di konsol.

3.5 Desain Prompt LLM

Desain *prompt* adalah salah satu keputusan metodologis yang paling berpengaruh terhadap kualitas respons model. Tiga tujuan harus dipenuhi sekaligus: mendapatkan kode perbaikan yang valid sintaksisnya, mendapatkan skor kepercayaan yang bisa di-

parse otomatis oleh *pipeline*, dan bisa dijalankan *zero-shot* tanpa contoh karena tidak ada *dataset* berlabel yang tersedia.

3.5.1 Struktur Prompt

Prompt terdiri dari dua bagian yang dikirim melalui endpoint `/api/chat: system message` dan *user message*. *System message*:

```
You are a PHP security expert and migration specialist. You analyze
PHP code for security vulnerabilities and provide secure PHP 8.x
replacements. Always follow output format instructions exactly as
given.
```

User message untuk setiap temuan:

```
TASK: Analyze the following PHP code for a [vulnerability_type]
vulnerability and provide a secure PHP 8.x replacement.
ISO/IEC 27001:2022 Controls: [relevant_controls]
Context: File: [filename], line [line_number],
        Semgrep rule: [rule_id], Finding: [description]
```

VULNERABLE CODE:

```
```php
[code_snippet]
```
```

Respond in English:

1. Explain the vulnerability and its security risk.
2. Provide the corrected PHP 8.x code in a ```php code block.

After your explanation and code fix, write exactly this on the last line:

CONFIDENCE: [number between 0 and 100]

Example last line: CONFIDENCE: 85

3.5.2 Keputusan Desain Prompt

Penyertaan kontrol ISO/IEC 27001:2022. Kontrol ISO yang relevan disertakan di setiap *prompt*. Ini bukan konteks dekoratif; tujuannya mendorong model menghasilkan

saran perbaikan yang mempertimbangkan kepatuhan standar, bukan sekadar memperbaiki sintaks.

Instruksi format di akhir pesan. Format `CONFIDENCE: [0-100]` diletakkan di bagian akhir dengan instruksi eksplisit termasuk contoh konkret. Model kecil cenderung lebih patuh pada instruksi yang dekat dengan akhir pesan.

Parsing skor berlapis. Karena tidak semua model mengikuti format yang diminta, `ai_engine.py` menggunakan enam pola *regex* berurutan untuk mengekstraksi skor: (0) tag XML `<confidence>`, (1) bilangan bulat dekat kata "confidence", (2) desimal `0.x` dekat "confidence", (3) persentase dekat "confidence", (4) desimal *bare* di mana saja, dan (5) frasa bahasa alami seperti "not sure" yang dipetakan ke nilai tetap. Jika semua gagal, dikembalikan nilai *default* 0,0.

Temperature 0,1. Dipilih untuk meminimalkan variasi acak antar percobaan. Setiap model dijalankan tiga kali per temuan dengan *prompt* identik, sehingga perbedaan hasil antar *run* mencerminkan variabilitas model yang sebenarnya.

Zero-shot tanpa contoh. Tidak ada contoh perbaikan kode disertakan karena penelitian ingin mengevaluasi kemampuan bawaan model dalam kondisi yang merepresentasikan penggunaan nyata. Ini juga memastikan semua model dibandingkan pada kondisi yang setara.

Skor kepercayaan sebagai metrik sekunder. Skor ini adalah penilaian mandiri dari model dan tidak bisa diverifikasi secara independen. Dalam analisis Bab IV, skor kepercayaan adalah metrik sekunder; metrik primer adalah validitas sintaks kode yang dihasilkan, diukur dengan `php -l`.

3.6 Desain Pemetaan ISO/IEC 27001:2022

Pemetaan temuan ke kontrol ISO/IEC 27001:2022 dilakukan secara berbasis aturan di dalam `iso_mapper.py`. Data diterima langsung sebagai objek Python *in-memory* dari `main.py`, tidak dibaca dari *file* JSON. Pilihan desain berbasis aturan ini disengaja agar hasilnya deterministik dan bisa diaudit, berbeda dari pendekatan berbasis AI yang hasilnya bisa berubah antar inferensi. Ada enam kontrol yang dipetakan, sebagaimana ditunjukkan pada Tabel 3.2.

Status akhir setiap kontrol ditentukan berdasarkan ada tidaknya temuan terbuka setelah *pipeline* selesai. `COMPLIANT` berarti tidak ada temuan yang dipetakan ke kontrol tersebut. `NON_COMPLIANT` berarti ada temuan Semgrep aktif. `PARTIAL` berarti hanya ada temuan PHPStan tanpa temuan Semgrep. Perlu dicatat bahwa `COMPLIANT` pada A.5.17 dan A.8.24 bukan berarti kode bebas masalah secara absolut, melainkan *ruleset* yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol itu pada dataset ini.

Tabel 3.2. Pemetaan sumber temuan ke kontrol ISO/IEC 27001:2022 Annex A

| Kontrol ISO | Judul | Sumber temuan yang dipetakan |
|-------------|--|--|
| A.5.17 | <i>Authentication Information</i> | Semgrep: <i>hardcoded credentials</i> |
| A.8.24 | <i>Use of Cryptography</i> | Semgrep: <code>md5()</code> / <code>sha1()</code> untuk kata sandi |
| A.8.25 | <i>Secure Development Lifecycle</i> | Semgrep (semua) + PHPStan ERROR/WARNING |
| A.8.26 | <i>Application Security Requirements</i> | Semgrep: SQL Injection, XSS |
| A.8.28 | <i>Secure Coding</i> | Semgrep (semua) + PHPStan ERROR |
| A.8.29 | <i>Security Testing in Development</i> | PHPStan ERROR/WARNING |

3.7 Dataset Studi Kasus

Studi kasus menggunakan 245 *file* PHP dari satu aplikasi web internal berbasis CodeIgniter 3.x yang dikelola oleh DTI UGM. Struktur direktorinya mengikuti konvensi standar CodeIgniter 3.x: `application/` untuk kode aplikasi (kontroler, model, *view*, *helper*, konfigurasi, layanan) dan `system/` untuk kode *framework*. Kode diperoleh melalui koordinasi langsung dengan DTI UGM dan hanya digunakan untuk analisis statis dalam lingkungan lokal, tanpa akses ke basis data produksi dan tanpa eksekusi kode.

Hanya *file* `.php` yang diproses; *file* lain seperti `.html`, `.js`, dan `.css` tidak disalin ke `output/` dan tidak dianalisis. Dataset mencerminkan karakteristik kode warisan yang umum: pencampuran logika bisnis, akses basis data, dan presentasi dalam *file* yang sama, serta beberapa pola kode PHP 5.x yang masih ada meski aplikasi secara keseluruhan berjalan di atas PHP 7.x.

Untuk eksperimen komparasi LLM, digunakan 11 temuan dari Semgrep *pre-scan*: 1 SQL Injection dan 10 SSRF (*Server-Side Request Forgery*). Ini adalah seluruh populasi temuan *pre-scan*, bukan subset yang dipilih secara subjektif.

3.8 Rancangan Eksperimen Komparasi Model LLM

Eksperimen komparasi dirancang dengan kondisi yang sepenuhnya terkontrol dan dilakukan terpisah dari pengujian *pipeline* utama menggunakan skrip evaluasi khusus.

3.8.1 Kondisi Eksperimen

- **Input:** 11 temuan Semgrep dari *pre-scan* dataset DTI UGM
- **Jumlah percobaan:** 3 *run* per temuan per model (33 percobaan per model, 165 total)
- **Temperature:** 0,1 untuk semua model

- **Prompt:** identik untuk semua model dan semua percobaan (*zero-shot*)
- **Model:** dijalankan lokal melalui Ollama tanpa modifikasi

3.8.2 Metrik Evaluasi

Format compliance rate mengukur persentase respons yang mengikuti format *prompt* yang diminta. Sebuah respons dianggap patuh jika mengandung blok kode PHP yang bisa diekstraksi, penjelasan bahasa alami, dan skor kepercayaan dalam format `CONFIDENCE : [0-100]`. Diukur otomatis oleh skrip `llm_comparison.py`. Metrik ini mencerminkan kemampuan model mengikuti instruksi, bukan kualitas kodenya.

Syntax validity rate mengukur persentase blok kode PHP yang diekstraksi dari respons dan lolos `php -l`. Diukur oleh `llm_quality_checker.py` secara objektif. Ini adalah metrik primer karena hasilnya konkret dan bisa diverifikasi secara independen. *Format compliance rate* adalah metrik sekunder.

3.8.3 Interpretasi Trade-off Antar Metrik

Kedua metrik bisa memberikan gambaran yang berbeda dan perlu diinterpretasikan hati-hati. Model yang patuh format tapi menghasilkan kode bermasalah secara sintaksis menunjukkan kemampuan mengikuti instruksi tapi kualitas kode rendah. Sebaliknya, model dengan kepatuhan format rendah tapi validitas sintaks tinggi menunjukkan model mengabaikan format instruksi namun kode yang berhasil diekstraksi berkualitas baik. Kedua kondisi ini punya implikasi praktis yang berbeda dan akan dibahas di Bab IV.

3.9 Keamanan dan Privasi Data

Kode PHP dari DTI UGM adalah aset institusi yang tidak boleh dikirimkan ke layanan eksternal. Kendala inilah yang menentukan arsitektur AI dalam penelitian ini. Daripada menggunakan API LLM berbasis *cloud*, penelitian ini menggunakan Ollama [22] sebagai *runtime* lokal sehingga seluruh inferensi terjadi di mesin pengembangan.

Tidak ada data yang dikirimkan ke *server* eksternal. Seluruh proses analisis berlangsung secara statis: tidak ada koneksi ke basis data produksi DTI UGM, tidak ada eksekusi kode PHP. Keputusan ini sejalan dengan ISO/IEC 27001:2022 [12] yang mensyaratkan pemisahan lingkungan pengembangan dan pengendalian akses terhadap aset informasi sensitif.

3.10 Metode Pengujian dan Evaluasi

Pengujian dilakukan dengan pendekatan *black-box*, fokus pada *output* setiap modul bukan detail implementasi internal. Ada lima metrik utama:

1. **Tingkat konversi Rector:** Persentase *file* PHP yang berhasil dikonversi tanpa kesalahan fatal Rector, dihitung per *file* dan secara keseluruhan.
2. **Validitas sintaks pasca-konversi:** Persentase *file* hasil konversi yang lolos php -1. Diukur secara objektif.
3. **Perbedaan temuan keamanan:** Selisih jumlah temuan Semgrep antara *pre-scan* dan *post-scan*. Temuan baru setelah konversi tidak otomatis diinterpretasikan sebagai regresi keamanan; bisa jadi merupakan kerentanan laten yang sebelumnya tersembunyi dan baru terlihat setelah transformasi kode.
4. **Performa model LLM:** Diukur menggunakan *format compliance rate* (sekunder) dan *syntax validity rate* (primer) sebagaimana dijelaskan dalam Subbab 3.8.
5. **Status kepatuhan ISO/IEC 27001:2022:** Status per kontrol (COMPLIANT, PARTIAL, atau NON_COMPLIANT) berdasarkan logika pemetaan dalam Subbab 3.6.

Untuk hasil PHPStan, jumlah kesalahan yang tinggi tidak langsung diinterpretasikan sebagai kegagalan *pipeline*. Kesalahan PHPStan pada kode warisan yang baru dimigrasi sebagian besar mencerminkan *technical debt* yang sudah ada sebelum migrasi, bukan akibat dari proses konversi. Perbedaan ini dibahas lebih rinci di Bab IV.

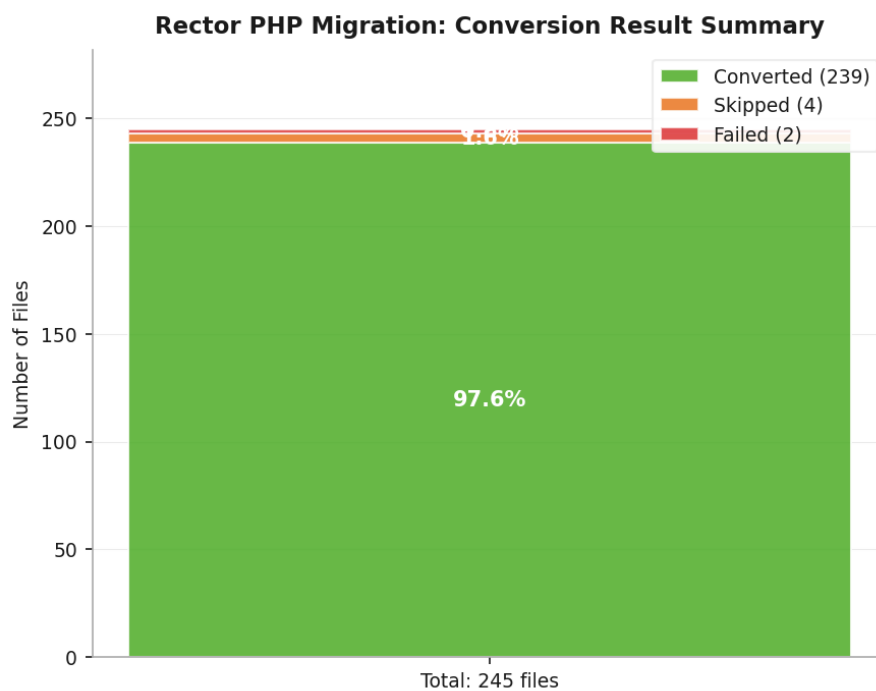
BAB IV

HASIL DAN DISKUSI

Bab ini menyajikan hasil implementasi *pipeline* pada studi kasus kode PHP milik DTI UGM beserta analisis dan diskusinya. Pembahasan dibagi menjadi lima bagian utama: hasil konversi Rector, hasil pemindaian keamanan Semgrep sebelum dan sesudah konversi, hasil validasi PHPStan, status kepatuhan ISO/IEC 27001:2022, dan hasil komparasi lima model LLM dalam kondisi *zero-shot* dengan *prompt* yang identik (Kondisi A).

4.1 Hasil Konversi Rector

Rector dijalankan terhadap 245 *file* PHP dari dataset DTI UGM dengan target versi PHP 8.3. Dari 245 *file* tersebut, 239 *file* berhasil dikonversi, 4 *file* dilewati (*skipped*), dan 2 *file* gagal diproses. Tingkat konversi keseluruhan adalah 97,6%, sebagaimana ditunjukkan pada Gambar 4.4.



Source: pipeline_result_20260425_120423.json | Skripsi Muhammad Farrel Akbar

Gambar 4.4. Ringkasan hasil konversi Rector terhadap 245 *file* PHP dataset DTI UGM

Karena dataset mengandung campuran kode PHP 5.x dan PHP 7.x, Rector menerapkan rantai *ruleset* kumulatif dari UP_TO_PHP_53 hingga UP_TO_PHP_83 secara otomatis dalam satu proses. Ini membuktikan bahwa *pipeline* dapat menangani kode dari berbagai era PHP sekaligus tanpa konfigurasi tambahan.

4.1.1 Contoh Transformasi Rector

Untuk memperlihatkan jenis-jenis perubahan yang dilakukan Rector secara konkret, berikut tiga contoh transformasi yang ditemukan pada dataset DTI UGM.

Transformasi 1 — Array sintaks pendek (ArrayShortSyntaxRector)

Pada `system/core/Security.php`, deklarasi *array* menggunakan sintaks lama diubah ke sintaks pendek PHP 5.4:

```
// Sebelum (PHP 7.x)
public $filename_bad_chars = array(
    '../', '<!--', '-->', '<', '>',
    '"', "'", '&', '$', '#',
);

protected $_never_allowed_str = array(
    'document.cookie' => '',
    'document.write'  => '',
);

// Sesudah (PHP 8.3)
public $filename_bad_chars = [
    '../', '<!--', '-->', '<', '>',
    '"', "'", '&', '$', '#',
];

protected $_never_allowed_str = [
    'document.cookie' => '',
    'document.write'  => '',
];
```

Transformasi 2 — Operator *null coalescing* (NullCoalescingRector)

Pada `system/helpers/url_helper.php`, pola `isset()` ternary diubah ke operator `??` yang lebih ringkas:

```
// Sebelum (PHP 7.x)
$atts[$key] = isset($attributes[$key]) ? $attributes[$key] : $val;

// Sesudah (PHP 8.3)
$atts[$key] = $attributes[$key] ?? $val;
```

Transformasi 3 — Fungsi `str_contains()` (`StrContainsRector`)

Pada `system/helpers/url_helper.php`, pola pencarian substring dengan `strpos()` diubah ke fungsi `str_contains()` yang diperkenalkan di PHP 8.0. Rector juga menambahkan *cast* (`string`) karena `$_SERVER['SERVER_SOFTWARE']` bertipe `mixed` di PHP 8.x:

```
// Sebelum (PHP 7.x)
if ($method === 'auto' && isset($_SERVER['SERVER_SOFTWARE'])
    && strpos($_SERVER['SERVER_SOFTWARE'], 'Microsoft-IIS') !== FALSE)

// Sesudah (PHP 8.3)
if ($method === 'auto' && isset($_SERVER['SERVER_SOFTWARE'])
    && str_contains((string) $_SERVER['SERVER_SOFTWARE'], 'Microsoft-
```

Ketiga transformasi ini mencerminkan kategori perubahan yang paling umum dilakukan Rector: modernisasi sintaks, penggunaan API PHP yang lebih baru, dan penambahan informasi tipe yang dibutuhkan PHP 8.x.

4.1.2 File yang Gagal Dikonversi

Kedua *file* yang gagal dikonversi mengalami *internal error* yang sama: `FiberNodeScopeResolver` menerima argumen bertipe `null` di mana seharusnya bertipe `PhpParser\Node`. Ini adalah *bug* pada versi Rector yang digunakan, bukan pada kode sumber DTI UGM.

File pertama, `captcha_helper.php`, memiliki pola ternary yang mengembalikan tipe berbeda (`GdImage` vs `resource`) tergantung pada hasil `function_exists()`. PHPStan *Fiber* yang digunakan Rector secara internal kehilangan referensi *node* AST saat mencoba menganalisis percabangan tipe yang berbeda ini.

File kedua, `form_helper.php`, memiliki lebih dari 15 definisi fungsi kondisional yang dibungkus `if (!function_exists(...))` di *scope file* yang sama. Dengan banyaknya *fiber* analisis yang dijalankan secara paralel per fungsi, terjadi kondisi *race* saat salah satu *callback* menerima *node* yang sudah selesai diproses oleh *fiber* lain.

Kedua *file* tetap tersedia di direktori `output/` sebagai salinan kode asli, sehingga proses tidak terhenti dan tidak ada data yang hilang.

Setelah konversi, seluruh *file* yang ada di direktori `output/` diuji validitas sintaksnya menggunakan `php -l`. Tidak ada *file* yang menghasilkan *parse error*, sehingga tingkat validitas sintaks pasca-konversi adalah 100%.

4.2 Hasil Pemindaian Keamanan Semgrep

4.2.1 Pre-Scan: Baseline Keamanan Kode Asli

Pemindaian Semgrep terhadap 241 *file* PHP di direktori `input/` menghasilkan 11 temuan yang terdiri dari 1 SQL Injection dan 10 SSRF (*Server-Side Request Forgery*). Semua 11 temuan berasal dari satu *file*: `Output.php`.

Temuan SQL Injection terdeteksi pada baris 768, di mana data dari `$_SERVER['QUERY_STRING']` mengalir langsung ke dalam konstruksi *cache path* tanpa sanitasi yang memadai. Semua 10 temuan SSRF terdeteksi pada beberapa baris di `Output.php` yang menggunakan nama *file* berbasis *input* pengguna untuk operasi *file system*, sehingga berpotensi dimanfaatkan untuk mengakses sumber daya yang tidak diinginkan.

Konsentrasi semua temuan pada satu *file* mencerminkan kondisi yang umum pada kode warisan: modul yang menangani output dan *cache* sering menjadi titik kerentanan karena banyaknya interaksi dengan data eksternal yang diproses tanpa validasi yang konsisten.

4.2.2 Post-Scan: Perbandingan Setelah Konversi

Pemindaian Semgrep terhadap direktori `output/` setelah konversi Rector menghasilkan 11 temuan, sama persis dengan *pre-scan*. Delta antara kedua pemindaian adalah nol. Distribusi per jenis kerentanan juga identik: 1 SQL Injection dan 10 SSRF, semuanya pada *file* yang sama.

Ini adalah hasil yang diharapkan dan sekaligus mengkonfirmasi dua hal. Pertama, proses konversi Rector tidak memperkenalkan kerentanan baru pada kode DTI UGM. Kedua, kerentanan yang sudah ada (SQL Injection dan SSRF di `Output.php`) tidak dapat diselesaikan oleh transformasi sintaksis berbasis AST karena memerlukan penulisan ulang logika secara semantik, yang merupakan domain AI Engine.

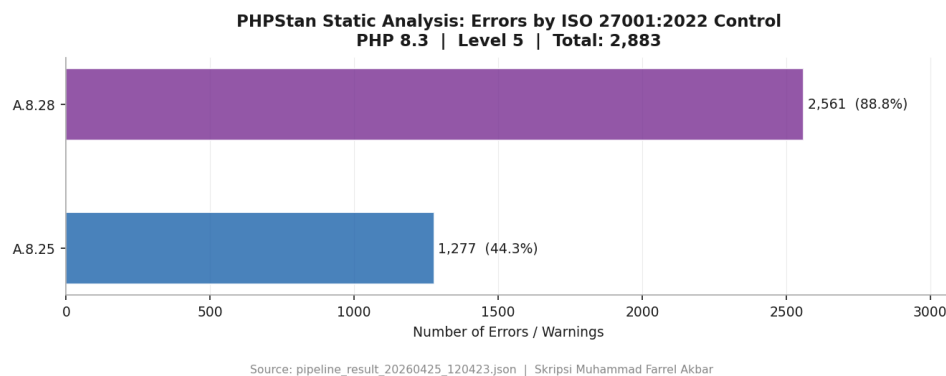
4.3 Hasil Validasi PHPStan

PHPStan level 5 dijalankan terhadap kode hasil konversi di direktori `output/` dengan target PHP 8.3. Hasilnya adalah 2.871 temuan yang terdiri dari 2.549 ERROR dan 322 WARNING, tersebar di 174 dari 241 *file* yang dianalisis.

Sebagian besar error yang ditemukan PHPStan berasal dari tiga kategori utama. Pertama, kelas dari *library* pihak ketiga yang tidak ditemukan, seperti `PhpOffice\PhpSpreadsheet` dan `Dotenv\Dotenv`. PHPStan tidak dapat menemukan kelas-kelas ini karena hanya kode sumber yang dianalisis, bukan dependensi lengkap aplikasi. Kedua, penggunaan nilai kembalian fungsi `void` sebagai nilai, misalnya `Result of method sendResponse() (void) is used`. Ini adalah pola yang umum pada kode warisan yang ditulis tanpa deklarasi tipe eksplisit. Ketiga, ketidakkonsistenan tipe seperti

Cannot access property \$role on array, yang mencerminkan penggunaan *array* sebagai pengganti objek bertipe tanpa definisi yang jelas.

Semua ini adalah manifestasi *technical debt* yang sudah ada sebelum migrasi, bukan kesalahan yang diperkenalkan oleh proses konversi. PHP 8.3 memiliki sistem tipe yang jauh lebih ketat dibandingkan PHP 7.x, sehingga masalah yang sebelumnya tidak terdeteksi karena PHP 7.x lebih permisif kini menjadi terlihat oleh PHPStan. Gambar 4.5 menunjukkan distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 yang dipetakan.



Gambar 4.5. Distribusi temuan PHPStan berdasarkan kontrol ISO/IEC 27001:2022 Annex A

Tingginya angka PHPStan justru membuktikan nilai dari komponen validasi dalam *pipeline* ini. Tanpa PHPStan, 2.871 potensi masalah tipe dan kompatibilitas ini tidak akan terdeteksi sampai kode dijalankan di lingkungan produksi, sesuai dengan temuan Strobl et al. [11] tentang pentingnya jaminan kualitas otomatis dalam transformasi kode skala besar.

4.4 Status Kepatuhan ISO/IEC 27001:2022

Berdasarkan pemetaan yang dilakukan oleh `iso_mapper.py`, status kepatuhan terhadap enam kontrol ISO/IEC 27001:2022 yang dipetakan adalah sebagaimana ditunjukkan pada Tabel 4.1.

Status keseluruhan adalah `NON_COMPLIANT`. Ini bukan hasil yang mengejutkan mengingat kondisi kode warisan yang dianalisis; justru ini adalah gambaran yang jujur tentang postur keamanan aplikasi sebelum dilakukan remediasi manual.

Dua kontrol yang berstatus `COMPLIANT`, yaitu A.5.17 dan A.8.24, perlu dipahami dalam konteks yang tepat. Status ini berarti *ruleset* Semgrep yang digunakan tidak menghasilkan temuan yang dipetakan ke kedua kontrol tersebut, bukan berarti kode bebas dari masalah autentikasi atau kriptografi secara absolut.

Kontrol A.8.29 berstatus `PARTIAL` karena *pipeline* sudah menjalankan penguji-

Tabel 4.1. Status kepatuhan ISO/IEC 27001:2022 hasil *pipeline* pada dataset DTI UGM

| Kontrol | Judul | Dasar Penilaian | Status |
|---------|--|---|---------------|
| A.5.17 | <i>Authentication Information</i> | Tidak ada temuan <i>hardcoded credentials</i> | COMPLIANT |
| A.8.24 | <i>Use of Cryptography</i> | Tidak ada temuan penggunaan <code>md5()</code> / <code>sha1()</code> untuk kata sandi | COMPLIANT |
| A.8.25 | <i>Secure Development Lifecycle</i> | 1.266 temuan PHPStan (masalah kompatibilitas dan kualitas kode) | NON_COMPLIANT |
| A.8.26 | <i>Application Security Requirements</i> | 1 temuan SQL Injection dari Semgrep | NON_COMPLIANT |
| A.8.28 | <i>Secure Coding</i> | Temuan Semgrep aktif dan 2.549 ERROR PHPStan | NON_COMPLIANT |
| A.8.29 | <i>Security Testing in Development</i> | 10 temuan SSRF dari Semgrep | PARTIAL |

an keamanan otomatis sebagai bagian dari proses, yang memenuhi sebagian persyaratan kontrol ini. Namun masih ada temuan terbuka, sehingga kontrol belum sepenuhnya terpenuhi. Laporan kepatuhan ini dihasilkan secara otomatis dan dapat langsung digunakan sebagai dokumentasi audit ISO/IEC 27001:2022 [12].

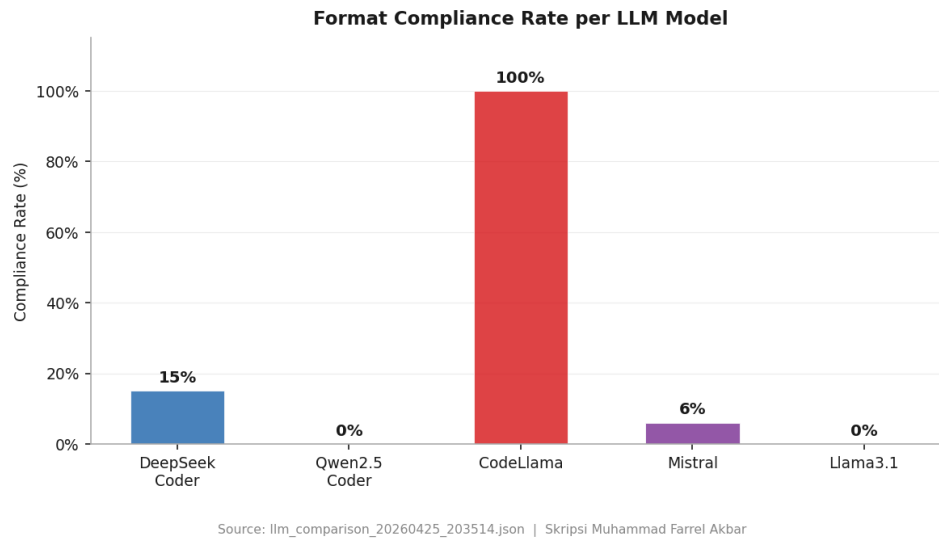
4.5 Hasil Komparasi Lima Model LLM (Kondisi A)

Eksperimen komparasi dijalankan dengan 11 temuan Semgrep *pre-scan* sebagai masukan, 3 *run* per temuan per model, *temperature* 0,1, dan *prompt* yang identik untuk semua model (*zero-shot*). Total percobaan adalah 165 (33 per model). Semua model dijalankan secara lokal melalui Ollama tanpa modifikasi.

4.5.1 Format Compliance Rate

Gambar 4.6 menunjukkan *format compliance rate* untuk masing-masing model.

CodeLlama adalah satu-satunya model yang secara konsisten mengikuti format

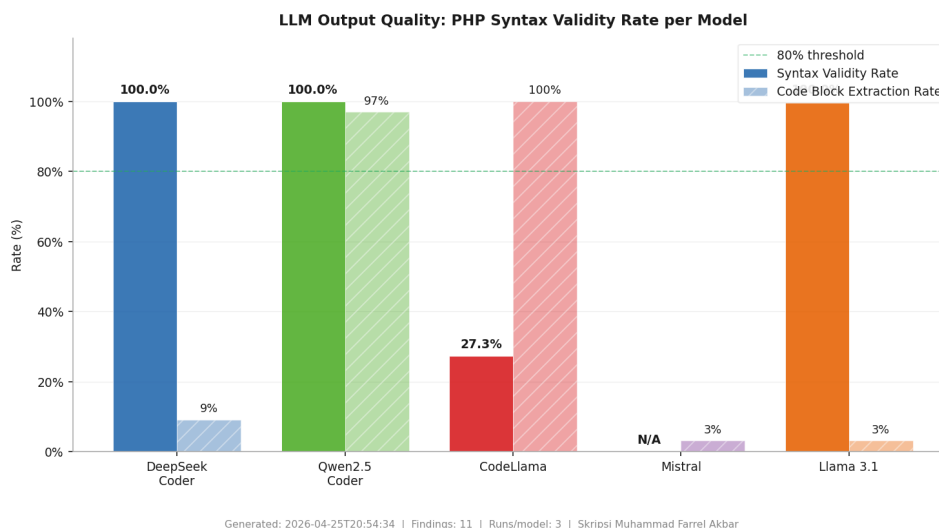


Gambar 4.6. *Format compliance rate* kelima model LLM pada Kondisi A (*zero-shot*, *prompt* identik)

instruksi *prompt* di semua 33 percobaan (100%). DeepSeek Coder mengikuti format di 5 dari 33 percobaan (15%). Mistral mengikuti format di 2 dari 33 percobaan (6%). Qwen2.5-Coder dan Llama 3.1 tidak menghasilkan satu pun respons yang memenuhi ketiga elemen format sekaligus.

4.5.2 Syntax Validity Rate

Gambar 4.7 menunjukkan *syntax validity rate* dan *code block extraction rate* untuk masing-masing model.



Gambar 4.7. *Syntax validity rate* dan *code block extraction rate* kelima model LLM

Hasilnya memperlihatkan gambaran yang berlawanan dengan metrik format. CodeLlama yang memiliki *format compliance* 100% ternyata hanya menghasilkan kode

PHP yang valid secara sintaksis pada 9 dari 33 blok kode yang berhasil diekstraksi, atau 27,3%. Ini berarti 24 dari 33 blok kode CodeLlama mengandung *parse error*.

Sebaliknya, Qwen2.5-Coder yang memiliki *format compliance* 0% berhasil mengekstraksi blok kode PHP dari 32 dari 33 percobaan (97%), dan seluruh 32 blok kode tersebut lolos pemeriksaan `php -l` (100%). Model ini mengabaikan instruksi format sepenuhnya, tetapi secara konsisten menghasilkan kode yang berkualitas baik.

Mistral berhasil mengekstraksi 1 blok kode dari 33 percobaan, tetapi blok tersebut tidak lolos `php -l` (0% validitas). DeepSeek Coder dan Llama 3.1 masing-masing mengekstraksi 3 dan 1 blok dengan validitas 100%, namun jumlah sampel yang terlalu kecil membuat angka ini tidak representatif secara statistik.

4.5.3 Statistik Ringkasan

Tabel 4.8 merangkum statistik lengkap untuk kelima model.

LLM Model Comparison -- Confidence Score Statistics

| Model | Mean | Std Dev | Min | Max | Compliance | Avg Time (s) |
|---------------------|-------|---------|-------|-------|------------|--------------|
| deepseek-coder:6.7b | 0.132 | 0.312 | 0.000 | 0.900 | 15% | 4.43 |
| qwen2.5-coder:7b | 0.000 | 0.000 | 0.000 | 0.000 | 0% | 9.98 |
| codellama:7b | 0.909 | 0.060 | 0.850 | 1.000 | 100% | 4.97 |
| mistral:7b | 0.058 | 0.227 | 0.000 | 0.950 | 6% | 7.28 |
| llama3.1:8b | 0.000 | 0.000 | 0.000 | 0.000 | 0% | 8.18 |

Source: llm_comparison_20260425_203514.json | Skripsi Muhammad Farrel Akbar

Gambar 4.8. Statistik lengkap komparasi kelima model LLM pada Kondisi A

CodeLlama memiliki *confidence score* rata-rata 0,909 dengan standar deviasi yang relatif kecil (0,054), mencerminkan konsistensi yang tinggi. Namun konsistensi ini tidak berkorelasi dengan kualitas kode: 72,7% kode CodeLlama tidak valid secara sintaksis. Ini mengkonfirmasi posisi metodologis dalam Bab III bahwa *confidence score self-reported* bukan ukuran kualitas kode.

Qwen2.5-Coder memiliki *confidence score* rata-rata 0,000 karena tidak pernah menghasilkan baris `CONFIDENCE: N`. Ketidakhadiran skor ini bukan tanda ketidakmampuan, melainkan refleksi dari gaya respons model yang mengabaikan instruksi format sambil tetap mengerjakan tugas utama dengan sangat baik. Waktu inferensi Qwen2.5-Coder (10,0 detik) paling lama dibanding model lain, kemungkinan karena model ini menghasilkan respons yang lebih panjang dan detail.

4.5.4 Analisis Kualitatif: Perbandingan Respons terhadap Temuan SQL Injection

Untuk memperlihatkan perbedaan perilaku model secara konkret, berikut perbandingan respons CodeLlama dan Qwen2.5-Coder terhadap temuan SQL Injection yang

sama dari Output.php baris 768.

Kode yang dikirimkan ke model adalah sebagai berikut:

```
$uri .= '?' . $_SERVER['QUERY_STRING'];
$cache_path .= md5($CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/'));

if ( ! @unlink($cache_path))
{
    log_message('error', 'Unable to delete cache file for ' . $uri);
    return FALSE;
}
```

Respons CodeLlama:

VULNERABILITY: SQL Injection Vulnerability

The vulnerability is caused by the manual construction of an SQL string, which allows user-controlled data to be incorporated into the query. This can lead to a variety of security risks, including unauthorized access, data tampering, and even complete system takeover.

SECURE PHP 8.x REPLACEMENT:

```
```php
 $uri = $_SERVER['QUERY_STRING'];
 $cache_path = md5($CI->config->item('base_url')
 . $CI->config->item('index_page')
 . ltrim($uri, '/'));

 if (! @unlink($cache_path)) {
 log_message('error', 'Unable to delete cache file for '
 . $uri);
 return FALSE;
 }
(blok kode tidak ditutup -- php -l gagal)
```

### **Respons Qwen2.5-Coder:**

### ### Explanation of Vulnerability

The provided PHP code constructs an SQL query string using user-controlled data (`$_SERVER['QUERY_STRING']`) without proper sanitization or parameterized queries. This can lead to a SQL Injection vulnerability...

### ### Secure PHP 8.x Code

To mitigate this vulnerability, you should use prepared statements with parameterized queries instead of manually constructing SQL strings. However, in this specific context, it seems that the code is not directly constructing an SQL query but rather using user-controlled data to construct a cache path. Therefore, we need to ensure that the user-controlled data is properly sanitized.

Here's the corrected PHP 8.x code:

```
```php
$uri .= '?' . urlencode($_SERVER['QUERY_STRING']);
$cache_path .= md5($CI->config->item('base_url')
    . $CI->config->item('index_page')
    . ltrim($uri, '/'));

if (!@unlink($cache_path)) {
    log_message('error', 'Unable to delete cache file for '
        . urlencode($uri));
    return FALSE;
}
```
(blok kode tidak ditutup dengan benar -- php -l gagal)
```

Dari perbandingan ini terlihat beberapa perbedaan yang signifikan. CodeLlama mengidentifikasi kerentanan dengan benar tapi solusinya tidak tepat: model hanya mengubah cara penulisan kode tanpa benar-benar mengatasi masalahnya. Kode yang dihasilkan masih menggunakan `$_SERVER['QUERY_STRING']` secara langsung tanpa sanitasi apapun.

Qwen2.5-Coder, sebaliknya, menunjukkan pemahaman kontekstual yang lebih dalam. Model ini menyadari bahwa meskipun Semgrep menandainya sebagai SQL In-

jection, kode tersebut sebenarnya membangun *cache path*, bukan kueri SQL. Saran perbaikannya menggunakan `urlencode()` untuk menghindari karakter berbahaya dalam *path*. Secara semantik ini lebih tepat, meski solusi yang ideal tetap perlu ditinjau oleh pengembang yang memahami konteks penuh aplikasi.

Kedua model gagal menutup blok kode PHP dengan benar pada *run* ini sehingga ekstraksi otomatis gagal. Ini konsisten dengan rendahnya *extraction rate* untuk kedua model pada temuan SQL Injection secara keseluruhan.

#### 4.5.5 Trade-off dan Implikasinya

Dari keseluruhan hasil Kondisi A, terdapat *trade-off* yang jelas antara kepatuhan format dan kualitas kode. Model yang paling patuh pada instruksi format (CodeLlama, 100%) menghasilkan kode dengan tingkat sintaksis error tertinggi (72,7% kode tidak valid). Model yang menghasilkan kode paling berkualitas (Qwen2.5-Coder, 97% *extraction rate* dan 100% validitas) mengabaikan instruksi format sepenuhnya.

Temuan ini memiliki implikasi langsung untuk desain sistem berbasis LLM lokal: jika tujuan utama adalah menghasilkan kode perbaikan yang dapat digunakan, metrik kualitas kode (*syntax validity rate*) adalah indikator yang jauh lebih relevan dibandingkan metrik kepatuhan format. Sistem evaluasi yang hanya mengandalkan kepatuhan format akan salah menilai model mana yang lebih berguna secara praktis.

Selain itu, *trade-off* ini juga menunjukkan bahwa kondisi *zero-shot* dengan *prompt* identik mungkin tidak optimal untuk semua model secara bersamaan. Model yang berbeda merespons desain instruksi yang berbeda dengan cara yang berbeda pula. Ini menjadi motivasi utama untuk Kondisi B, di mana setiap model akan menerima *prompt* yang dioptimasi berdasarkan karakteristik dan kelemahan yang terobservasi pada Kondisi A.

### 4.6 Diskusi dan Keterbatasan

#### 4.6.1 Keterbatasan Dataset

Dataset yang digunakan terdiri dari satu aplikasi berbasis CodeIgniter 3.x milik DTI UGM. Dari 245 *file*, 237 diklasifikasikan sebagai *unknown* oleh Rector karena merupakan *file framework* CodeIgniter yang tidak mengandung pola yang perlu ditransformasi. Hal ini membatasi seberapa jauh hasil penelitian ini bisa digeneralisasi ke jenis aplikasi PHP lain, terutama yang tidak menggunakan *framework* atau yang menggunakan *framework* berbeda.

Dari sisi input eksperimen LLM, 10 dari 11 temuan adalah SSRF yang berasal dari satu pola kode yang berulang di `Output.php`. Homogenitas input ini berarti eksperimen komparasi pada dasarnya menguji respons model terhadap dua jenis kerentanan dengan variasi terbatas. Hasil komparasi perlu diinterpretasikan dalam konteks keterba-

tasannya ini.

#### **4.6.2 Keterbatasan Teknis Pipeline**

AI Engine hanya memproses temuan dengan prioritas 1–3 dan membatasi potongan kode yang dikirim ke model pada 2.000 karakter pertama. Untuk *file* PHP yang panjang, konteks yang diterima model terpotong sehingga saran perbaikan mungkin tidak mempertimbangkan keseluruhan logika fungsi. Sebagaimana terlihat pada contoh respons Qwen2.5-Coder di subbab sebelumnya, model dapat membuat inferensi yang tepat tentang konteks kode meski dengan informasi yang terbatas, namun tidak selalu bisa dipastikan.

Selain itu, validasi menggunakan `php -l` hanya memeriksa kebenaran sintaksis, bukan kebenaran semantik. Kode yang lolos `php -l` bisa saja masih mengandung kerentanan yang tidak tertangani atau logika yang tidak ekuivalen dengan kode aslinya. Pengujian fungsional terhadap kode hasil perbaikan AI bukan bagian dari cakupan penelitian ini.

#### **4.6.3 Interpretasi Confidence Score**

Tidak ada korelasi yang jelas antara *confidence score* yang dilaporkan model dan kualitas kode yang dihasilkan. CodeLlama melaporkan kepercayaan diri rata-rata 90,9% tetapi 72,7% kode yang dihasilkannya tidak valid secara sintaksis. Qwen2.5-Coder tidak melaporkan skor sama sekali tetapi menghasilkan kode yang hampir selalu berkualitas baik. Temuan ini konsisten dengan literatur yang menunjukkan bahwa model LLM berukuran kecil umumnya tidak terkalibrasi dengan baik dalam menilai output mereka sendiri [13] [15].

## **BAB V**

### **TAMBAHAN (OPSIONAL)**

Anda boleh menambahkan Bab jika diperlukan. Jumlah Bab tidak harus sesuai dengan *template*.

Bab tambahan ini diperlukan jika hasil penelitian untuk menjawab tujuan cukup panjang atau terdiri dari banyak sub bab. Mahasiswa boleh menjawab 1 tujuan penelitian dengan 1 bab.



## **BAB VI**

### **KESIMPULAN DAN SARAN**

#### **6.1 Kesimpulan**

Kesimpulan dapat diawali dengan apa yang dilakukan dengan tugas akhir ini lalu dilanjutkan dengan poin-poin yang menjawab tujuan penelitian, apakah tujuan sudah tercapai atau belum, tentunya berdasarkan data ataupun hasil dari Bab pembahasan sebelumnya. Dalam beberapa hal, kesimpulan dapat juga berisi tentang temuan/*findings* yang Anda dapatkan setelah melakukan pengamatan dan atau analisis terhadap hasil penelitian.

Kesimpulan menjawab seberapa jauh rumusan masalah tercapai berdasarkan hasil penelitian. Semua rumusan masalah harus disimpulkan berdasarkan data penelitian.

#### **6.2 Saran**

Saran berisi hal-hal yang bisa dilanjutkan dari penelitian atau skripsi ini, yang belum dilakukan karena batasan permasalahan. Saran bukan berisi saran kepada sistem atau pengguna, tetapi saran diberikan kepada aspek penelitian yang dapat dikembangkan dan ditambahkan di penelitian atau skripsi selanjutnya.

**Catatan: Mahasiswa perlu melihat sinkronisasi antara rumusan masalah, tujuan, metode, hasil penelitian, dan kesimpulan.**

## DAFTAR PUSTAKA

- [1] The PHP Group, “PHP: History of PHP,” php.net, 2024, [Online]. Available: <https://www.php.net/manual/en/history.php>. [Accessed: Apr. 2026].
- [2] W3Techs, “Usage statistics and market share of PHP for websites,” W3Techs Web Technology Surveys, 2026, [Online]. Available: <https://w3techs.com/technologies/details/pl-php>. [Accessed: Apr. 8, 2026].
- [3] The PHP Group, “PHP: Supported versions,” php.net, 2025, [Online]. Available: <https://www.php.net/supported-versions.php>. [Accessed: Apr. 2026].
- [4] National Institute of Standards and Technology, “National vulnerability database (NVD),” NIST, 2024, [Online]. Available: <https://nvd.nist.gov/>. [Accessed: Apr. 2026].
- [5] OWASP Foundation, “OWASP top 10:2021 — the ten most critical web application security risks,” OWASP Foundation, 2021, [Online]. Available: <https://owasp.org/Top10/>. [Accessed: Apr. 2026].
- [6] E. Merlo, D. Letarte, and G. Antoniol, “Automated protection of PHP applications against SQL-injection attacks,” in *Proc. 11th European Conf. on Software Maintenance and Reengineering (CSMR)*, Amsterdam, Netherlands, 2007, pp. 191–202.
- [7] Rector, “Rector — instant upgrades and automated refactoring,” getrector.com, 2024, [Online]. Available: <https://getrector.com>. [Accessed: Apr. 2026].
- [8] D. Chen, D. Lawler, Y. Zhang, P. Kuchhal, D. Westcott, and G. Yang, “AST-based source code migration through symbols replacement,” in *Proc. 2022 IEEE Asia-Pacific Conf. on Computer Science and Data Engineering (CSDE)*, Brisbane, Australia, 2022, pp. 1–6.
- [9] Semgrep, “Semgrep — lightweight static analysis for many languages,” semgrep.dev, 2024, [Online]. Available: <https://semgrep.dev>. [Accessed: Apr. 2026].
- [10] PHPStan, “PHPStan — php static analysis tool,” phpstan.org, 2024, [Online]. Available: <https://phpstan.org>. [Accessed: Apr. 2026].
- [11] S. Strobl, C. Zoffi, C. Haselmann, M. Bernhart, and T. Grechenig, “Automated code transformations: Dealing with the aftermath,” in *Proc. 2020 IEEE 27th Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, London, ON, Canada, 2020, pp. 627–631.
- [12] International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 27001:2022 Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems: Requirements*, International Organization for Standardization and International Electrotechnical Commission Std., 2022.
- [13] V. Akuthota, R. Kasula, S. T. Sumona, M. Mohiuddin, M. T. Reza, and M. M. Rahman, “Vulnerability detection and monitoring using LLM,” in *Proc. 2023 IEEE*

*9th Int. Women in Engineering (WIE) Conf. on Electrical and Computer Engineering (WIECON-ECE)*, Thiruvananthapuram, India, 2023, pp. 309–314.

- [14] C. Li, B. Guan, Q. Zheng, B. Liu, and Z. Zhang, “Software vulnerability detection based on LLM,” in *Proc. 2025 IEEE 7th Advanced Information Management, Communicates, Electronic and Automation Control Conf. (IMCEC)*, Chongqing, China, 2025, pp. 1534–1539.
- [15] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, Vienna, Austria, 2025, [Online]. Available: <https://arxiv.org/abs/2405.17238>.
- [16] J. Stümpfle, D. Atray, N. Jazdi, and M. Weyrich, “Large language model assisted transformation of software variants into a software product line,” in *Proc. 2025 IEEE/ACM 22nd Int. Conf. on Software and Systems Reuse (ICSR)*, Ottawa, ON, Canada, 2025, pp. 12–20.
- [17] DeepSeek-AI, “DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence,” *arXiv preprint arXiv:2401.14196*, 2024, [Online]. Available: <https://arxiv.org/abs/2401.14196>.
- [18] B. Hui *et al.*, “Qwen2.5-Coder technical report,” *arXiv preprint arXiv:2409.12186*, 2024, [Online]. Available: <https://arxiv.org/abs/2409.12186>.
- [19] B. Rozière *et al.*, “Code Llama: Open foundation models for code,” *arXiv preprint arXiv:2308.12950*, 2024, [Online]. Available: <https://arxiv.org/abs/2308.12950>.
- [20] A. Q. Jiang *et al.*, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023, [Online]. Available: <https://arxiv.org/abs/2310.06825>.
- [21] A. Dubey *et al.*, “The Llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024, [Online]. Available: <https://arxiv.org/abs/2407.21783>.
- [22] Ollama, “Ollama — get up and running with large language models locally,” [ollama.com](https://ollama.com), 2024, [Online]. Available: <https://ollama.com>. [Accessed: Apr. 2026].
- [23] Z. Cai, L. Zhao, X. Wang, X. Yang, J. Qin, and K. Yin, “A pattern-based code transformation approach for cloud application migration,” in *Proc. 2015 IEEE 8th Int. Conf. on Cloud Computing (CLOUD)*, 2015, pp. 33–40.

Catatan: Daftar pustaka adalah apa yang dirujuk atau disitasi, bukan apa yang telah dibaca, jika tidak ada dalam sitasi maka tidak perlu dituliskan dalam daftar pustaka.